EH/Unwind Analyzer

Теоретическая база с нуля

Третье переработанное издание: цельный лекционный учебник по PE, адресам, Windows x64 EH/Unwind, статике, динамике, отчетам и Ghidra

Как читать эту версию
Это не набор одинаковых уроков. Материал построен как университетский курс: от простого опыта работы с файлом до строгого анализа unwind metadata и защиты выводов. Читай медленно, связывай термины между собой и проверяй каждый важный тезис на отчете или в Ghidra.

Предисловие. Почему этот учебник переписан заново

Предыдущая попытка сделать расширенное издание была неправильной: страницы выглядели слишком одинаково. Настоящий учебник не должен быть набором карточек с одинаковыми заголовками. Он должен вести мысль. Поэтому эта версия построена как связный университетский курс: сначала простое объяснение, затем строгая модель, затем проект, затем отчеты, Ghidra, динамика и защита вывода.

Здесь сохраняется требование глубины, но меняется подача. Мы будем говорить живо: иногда через бытовую аналогию, иногда через точную структуру, иногда через ошибку новичка, иногда через вопрос преподавателя. Цель не в том, чтобы дочитать 500 страниц. Цель в том, чтобы после чтения в голове сложилась карта.

Том 0. Как сформировано оглавление: источники, традиция и границы этой книги

Фундаментальный учебник нельзя строить только из желания "объяснить поподробнее". Нужно понимать, на каких источниках держится предмет. У этой книги есть три опоры. Первая - официальная документация Microsoft и Intel, потому что PE, x64 ABI, unwind метаданные, Debug API и обработка исключений являются системными контрактами. Вторая - классическая литература по реверс-инжиниринг и анализ вредоносного ПО, потому что там выработана педагогика: от статического наблюдения к динамическому, от инструмента к выводу, от байта к смыслу. Третья - исходный код EH/Unwind Analyzer, потому что книга должна объяснять не абстрактный PE вообще, а конкретный проект и его отчеты.

Важно: эта книга не пересказывает чужие главы и не копирует защищенные тексты. Она использует источники как карту темы. Официальная документация дает точные определения и ограничения. Книги по реверсу дают порядок обучения и практический стиль. Проект дает живой предмет: парсер, декодер, эвристики, динамический сборщик, отчеты, Ghidra CSV и regression образцы.

Источники и как они повлияли на книгу

Спецификация PE/COFF Microsoft: как Windows понимает исполняемый файл

Этот документ важен не как ссылка для библиографии, а как основа всей координатной системы проекта. Он объясняет, почему исполняемый файл Windows состоит из заголовков, секций и каталогов данных, а не является просто набором команд процессора.

Для EH/Unwind Analyzer особенно важны три идеи: файл на диске отличается от образа в памяти, адреса требуют перевода, а каталог исключений ведет к служебным таблицам x64. Без этой базы

невозможно честно объяснить, почему один адрес можно найти в HTML-отчете, в Ghidra и в байтах файла.

Официальное название источника: Microsoft PE/COFF Specification.

Ссылка для самостоятельной проверки: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format

Соглашение о вызовах Microsoft x64: как функции договариваются между собой

Соглашение о вызовах объясняет, как функции передают аргументы, кто сохраняет регистры, почему стек должен быть выровнен и зачем существует область, которую часто называют теневым местом под аргументы. Это правила поведения функций на машинном уровне.

Unwind-метаданные нельзя понять отдельно от этих правил. Если функция изменила стек или сохранила регистр, Windows должна знать, как восстановить состояние при исключении. Поэтому соглашение о вызовах является не соседней темой, а фундаментом для UNWIND_INFO.

Официальное название источника: Microsoft x64 calling convention.

Ссылка для самостоятельной проверки: https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention

Обработка исключений Microsoft x64: почему существуют RUNTIME_FUNCTION и UNWIND_INFO

Эта документация связывает исключение, стек, диапазон функции и служебные структуры Windows x64 в один механизм. Она показывает, зачем таблица RUNTIME_FUNCTION хранит начало функции, конец функции и адрес UNWIND_INFO.

Именно отсюда берется строгий смысл проверок проекта. Если запись выходит за границы, если UNWIND_INFO не декодируется, если флаги ведут к странному обработчику, анализатор не фантазирует, а проверяет нарушение ожидаемого договора Windows

x64.

Официальное название источника: Microsoft x64 обработка исключений.

Ссылка для самостоятельной проверки: https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64

RtlAddFunctionTable: как программа добавляет таблицу функций во время выполнения

Эта функция показывает, что таблица функций может появиться не только из .pdata внутри файла. Программа может зарегистрировать таблицу во время выполнения, и Windows начнет учитывать ее при раскрутке стека.

Для динамики проекта это принципиально: статический отчет видит файл, а динамический запуск может увидеть регистрацию время выполнения-таблицы. Такое событие важно не потому, что оно автоматически вредоносно, а потому что оно меняет карту функций во время работы процесса.

Официальное имя API: RtlAddFunctionTable.

Ссылка для самостоятельной проверки: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdfunctiontable

RtlInstallFunctionTableCallback: как Windows запрашивает таблицу функций через обратный вызов

Этот API идет еще дальше: таблица функций может предоставляться через обратный вызов. То есть Windows при необходимости обращается к коду программы, чтобы получить сведения о функциях.

Для учебника это хороший пример динамического поведения: часть картины может отсутствовать в статическом файле и появляться только при запуске. Поэтому проект отслеживает не только байты, но и события регистрации таблиц.

Официальное имя API: RtlInstallFunctionTableCallback.

Ссылка для самостоятельной проверки: https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtlinstallfunctiontablecallback

События отладки Microsoft: как отладчик наблюдает процесс

Документация по событиям отладки объясняет, как внешний наблюдатель получает сообщения о создании процесса, загрузке модулей, исключениях и завершении. Это язык, на котором динамический сборщик разговаривает с Windows.

Когда в отчете появляются CREATE_PROCESS, EXCEPTION или EXIT_PROCESS, это не произвольные строки лога. Это следы событий отладочного интерфейса. Поэтому динамический отчет надо читать как протокол эксперимента, а не как вторую версию статической таблицы.

Официальная группа источников: Debugging События and Debugging Functions.

Ссылка для самостоятельной проверки: https://learn.microsoft.com/en-us/windows/win32/debug/debugging-события

VEH и UEF API Microsoft: как программа регистрирует обработчики исключений

Vectored Exception Обработчик и Unhandled Exception Filter описывают два способа зарегистрировать код, который участвует в обработке исключений. На русском их удобно называть векторным обработчиком и фильтром необработанного исключения.

Для проекта это важно потому, что обработчик является исполняемым адресом. Если он зарегистрирован, находится в необычной секции или реально срабатывает, это уже не абстрактный термин, а конкретное наблюдение, которое можно показать в отчете и Ghidra.

Официальные имена API: AddVectoredExceptionHandler и SetUnhandledExceptionFilter.

Ссылка для самостоятельной проверки: https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-addvectoredexceptionhandler

Руководства Intel по архитектуре x64: что реально делает процессор

Windows задает формат PE и правила исключений, но сами инструкции выполняет процессор. Руководства Intel объясняют регистры, стек, переходы, вызовы функций, возврат и эффекты инструкций.

Без этой базы можно неправильно читать дизассемблер. Например, изменение RSP является не просто строкой в Ghidra, а изменением вершины стека. Сохранение регистра важно потому, что соглашение о вызовах назначает ответственность за его восстановление.

Официальное название источника: Intel 64 and IA-32 Architectures Software Developer's Manuals.

Ссылка для самостоятельной проверки: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

Ghidra от NSA: среда ручной проверки адресов, кода и данных

Ghidra нужна в книге как инструмент ручной проверки, а не как безошибочный судья. Она помогает перейти к адресу, увидеть инструкции, секцию, ссылки и приблизительную модель функции.

Если отчет показывает representative RVA или адрес обработчика, Ghidra помогает проверить, что там находится. Но границы функций, восстановленный C-подобный код и имена, которые предлагает инструмент, надо воспринимать как результат анализа, а не как исходную истину.

Официальное название инструмента: NSA Ghidra Software Reverse Engineering Framework.

Ссылка для самостоятельной проверки: https://github.com/NationalSecurityAgency/ghidra

Practical Malware Analysis: традиция статического и динамического разбора

Эта книга важна прежде всего методикой. Она учит разделять базовый статический анализ, базовый динамический анализ, дизассемблирование, отладку и лабораторную проверку.

В этом учебнике взята не чужая формулировка, а педагогическая логика: сначала наблюдение, потом инструмент, потом эксперимент, потом осторожный вывод. Именно так надо читать отчеты EH/Unwind Analyzer.

Название источника: Practical Malware Analysis.

Ссылка для самостоятельной проверки: https://nostarch.com/malware

Eldad Eilam, Reversing: широкий взгляд на низкоуровневый анализ

Эта книга полезна широтой взгляда: реверс-инжиниринг не сводится к одной кнопке дизассемблера. Это восстановление смысла программы через форматы файлов, Windows, машинный код, инструменты и ручную проверку.

В нашем учебнике этот подход проявляется в том, что PE, стек, исключения, Ghidra и отчеты не разбросаны по отдельным островам. Они собираются в одну цепочку доказательства.

Название источника: Reversing: Secrets of Reverse Engineering.

Ссылка для самостоятельной проверки: https://en.wikipedia.org/wiki/Reversing:_Secrets_of_Reverse_Engineering

Dennis Yurichev, Reverse Engineering for Beginners: путь новичка через ассемблер и компилятор

Этот источник полезен тем, что уважает новичка. Он показывает, как постепенно привыкнуть к ассемблеру, машинному представлению и следам компилятора.

Для текущей книги это особенно важно: читатель должен не заучить аббревиатуры, а начать видеть, как высокоуровневая программа превращается в байты, адреса, функции и служебные структуры Windows.

Название источника: Reverse Engineering for Beginners.

Ссылка для самостоятельной проверки: https://beginners.re/

Почему главы идут именно в таком порядке

Лучшие учебники не начинают с самого красивого результата. Они начинают с того, без чего результат будет магией. Поэтому здесь сначала идут байты, числа, файл, память и адреса. Потом появляется PE как договор между компоновщиком и загрузчиком. Потом - x64 ABI, потому что без стека, регистров, prolog и epilog невозможно понять unwind метаданные. Только после этого есть смысл говорить о RUNTIME_FUNCTION, UNWIND_INFO, обработчики, chained info и dynamic таблицы функций.

После системной базы идет метод реверс-инжиниринг: как смотреть на бинарник, чем отличается статический анализ от динамическая проверка, зачем нужен Ghidra, почему признак не является приговором. Затем книга переходит к самому проекту: как src/main.c запускает pipeline, как util.c защищает чтение, как pe.c строит модель PE, как unwind.c декодирует метаданные, как heuristics.c превращает факты в свидетельства, как dynamic_win.c наблюдает время выполнения и как report.c делает вывод проверяемым.

Такой порядок выбран сознательно. Если раньше времени дать человеку 27 признаков, он будет заучивать названия. Если сначала дать ему файловое смещение, RVA, секция, кадр стека, UNWIND_INFO и свидетельства, признаки становятся естественным следствием.

Как читать источники после этой книги

Официальная документация Microsoft написана точно, но не обязана быть учебником для нулевого читателя. Она говорит, как устроено поле или API, но не всегда объясняет, почему новичок путается. Книги по реверс-инжиниринг часто дают широкий опыт, но не фокусируются на конкретном EH/Unwind Analyzer. Поэтому эта книга выполняет роль моста: она подготавливает язык, карту и вопросы, после которых Microsoft Learn, Intel manuals и Ghidra docs читаются уже не как стена терминов, а как знакомая техническая территория.

Том 0-A. Большая вводная теория: как из программы получается объект анализа

Эта глава добавлена как медленный вход в предмет. Она не требует, чтобы читатель уже знал ассемблер, Windows API или формат PE. Ее задача - собрать в голове первую цельную карту: от обычной программы к исполняемому файлу, от файла к памяти, от памяти к процессу, от процесса к исключению, от исключения к unwind-метаданным, от метаданных к отчету и ручной проверке.

Читать ее нужно не как справочник, а как лекцию. Здесь важны связи. Байты важны не сами по себе, а потому что из них строятся структуры. Адреса важны не сами по себе, а потому что переводят нас между файлом, образом и процессом. Ghidra важна не сама по себе, а потому что помогает проверить конкретный адресный вопрос. Отчет важен не сам по себе, а потому что превращает низкоуровневые факты в воспроизводимую форму.

1. Что такое программа до того, как она стала EXE

Для человека программа обычно начинается с исходного текста: функций, переменных, условий и комментариев. Для компьютера такой текст сам по себе еще не является исполняемой программой. Между текстом и запуском стоит цепочка превращений: компилятор переводит идеи в машинные инструкции, компоновщик собирает части вместе, а загрузчик Windows готовит образ к выполнению в памяти.

Новичок часто думает, что EXE - это просто программа в сжатом виде. Это неверная картина. EXE ближе к контейнеру с инструкциями, данными, таблицами, адресами, правилами загрузки и служебной информацией. В этом контейнере есть не только то, что написал программист, но и то, что нужно операционной системе: где начинается код, где данные, какие библиотеки нужны, какие диапазоны функций существуют, как восстанавливать стек при

исключении.

Для EH/Unwind Analyzer это главный стартовый тезис. Проект исследует не намерения автора программы напрямую, а следы, которые остались в исполняемом файле и во время запуска. Поэтому первая зрелая привычка реверс-инженера - не спрашивать сразу 'что программа хотела сделать', а спрашивать 'какой слой я сейчас вижу и по каким правилам он устроен'.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

2. Почему бинарник нельзя читать как обычный текст

Текстовая книга хранит буквы, и человек сразу видит слова. Бинарник хранит байты, а смысл байта зависит от места и контекста. Один и тот же набор байтов может быть числом, частью инструкции, адресом, строкой, выравниванием или полем структуры. Поэтому реверс-инжиниринг начинается не с чтения, а с разметки: нужно понять, где какой слой данных начинается и заканчивается.

Формат PE помогает внести порядок. Он говорит: вот заголовки, вот таблица секций, вот каталог данных, вот место, где можно найти таблицу исключений. Но даже формат не отменяет осторожность. Файл может быть поврежден, намеренно изменен, необычно собран или просто не похож на учебный пример. Значит, анализатор обязан проверять границы, размеры и перевод адресов, а не верить каждому полю.

Когда в отчете написано, что адрес выходит за границы или структура не декодируется, это не эмоция инструмента. Это результат дисциплины чтения бинарника: сначала проверить, что байты

действительно доступны, потом понять, какая структура там ожидается, и только потом делать вывод о подозрительности.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

3. Файл, образ и процесс: три состояния одного объекта

Один и тот же EXE можно увидеть в трех состояниях. Первое состояние - файл на диске. Здесь все измеряется смещениями от начала файла. Второе состояние - образ, который Windows подготовила для выполнения. Здесь важны адреса внутри образа, секции, права и база загрузки. Третье состояние - живой процесс. Здесь появляются потоки, реальные виртуальные адреса, загруженные библиотеки, исключения, обработчики и события отладки.

Путаница между этими состояниями разрушает анализ. Если число выглядит как адрес, еще не ясно, адрес это в файле, в образе или в процессе. Если в статическом отчете указан RVA, его нужно переводить в координаты Ghidra и при необходимости в файловое смещение. Если в динамическом логе указан адрес во время выполнения, нужно помнить о базе загрузки и ASLR.

Учебная картина такая: файл дает устойчивые байты, образ дает системную карту, процесс дает наблюдение поведения. Статика сильна тем, что видит всю доступную структуру файла. Динамика сильна тем, что показывает, что реально произошло в конкретном запуске. Ни один слой не отменяет другой.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена,

которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

4. Компилятор, компоновщик и загрузчик как три автора следов

В реверс-инжиниринге важно понимать, что многие байты в программе написал не человек вручную. Компилятор выбрал инструкции и построил пролог функции. Компоновщик разложил секции и служебные таблицы. Загрузчик Windows отобразил секции в память, применил необходимые настройки и подготовил процесс. Поэтому бинарник является результатом работы нескольких участников.

Если проект находит необычную секцию или странную unwind-структуру, это не означает автоматически злой умысел. Нужно спросить, кто мог создать этот след. Иногда это компилятор, иногда упаковщик, иногда защитный инструмент, иногда ручная модификация, иногда тестовый образец из лабораторного набора. Сильный анализ не угадывает автора, а собирает признаки.

Для защиты диплома это особенно важно. Формулировка 'анализатор нашел вредоносность' слабая. Формулировка 'анализатор нашел нарушение ожидаемой структуры Windows x64 unwind метаданные, указал адрес, секцию и проверяемое основание' намного сильнее и честнее.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

5. Что такое функция в бинарнике

В исходном коде функция видна глазами: есть имя, фигурные скобки и тело. В бинарнике таких удобных границ может не быть. Функция становится диапазоном инструкций, который нужно восстановить по символам, дизассемблированию, служебным таблицам, соглашению о вызовах и контексту.

На Windows x64 важную роль играет таблица RUNTIME_FUNCTION. Она дает системный взгляд: от какого адреса до какого адреса находится функция и где лежит UNWIND_INFO. Ghidra может строить свою разметку функций по инструкциям и эвристикам. Иногда эти взгляды совпадают, иногда расходятся. Расхождение не нужно прятать, его нужно объяснять.

EH/Unwind Analyzer работает именно с тем, как Windows должна понимать диапазоны функций для обработки исключений. Поэтому он не заменяет весь анализ программы, но очень хорошо подсвечивает отдельный критичный слой: официальную карту функций и восстановления стека.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

6. Стек как память разговора между функциями

Стек можно представить как рабочую тетрадь потока выполнения. Когда функция вызывается, там появляются адрес возврата, временные данные, сохраненные регистры и место для локальной работы. Когда функция завершается, стек должен вернуться в согласованное состояние. Если этого не происходит, выполнение ломается.

На x64 стек особенно важен для исключений. Исключение может произойти не в удобный момент, а внутри функции, после изменения указателя стека и сохранения регистров. Windows должна понять, как аккуратно перейти назад к вызывающей функции. Для этого ей нужна не догадка, а карта действий функции.

Этой картой и является UNWIND_INFO. Она не описывает всю логику функции. Она описывает только то, что важно для восстановления состояния: какие операции со стеком и регистрами были сделаны, есть ли обработчик, есть ли цепочка к другой записи. Поэтому маленькая служебная структура может иметь большое значение.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

7. Исключение как управляемый сбой, а не просто ошибка

Слово 'исключение' легко понять бытово: что-то пошло не так. Но в Windows исключение является управляемым событием. Система получает контекст потока, смотрит, где произошло событие, ищет подходящую информацию о функции и решает, кому дать шанс обработать ситуацию.

Существуют разные участники этого процесса: структурированная обработка исключений, векторные обработчики, фильтр необработанного исключения, таблицы функций времени выполнения. Они не являются одной и той же вещью. Одни связаны с диапазонами функций и unwind-метаданными, другие регистрируются как обратный вызов и участвуют в маршруте обработки.

Динамический отчет проекта ценен потому, что может увидеть не только статическую структуру, но и факты регистрации или срабатывания. Если виден RtlAddFunctionTable, значит программа во время выполнения добавляла таблицу функций. Если виден AddVectoredExceptionHandler, значит регистрировался векторный обработчик. Эти наблюдения нужно читать как события эксперимента.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

8. Почему метаданные бывают важнее видимого кода

Новичок часто хочет смотреть только дизассемблер и декомпилятор. Это понятно: инструкции похожи на действие, а таблицы кажутся скучными. Но операционная система часто принимает решения именно по служебным данным. Если метаданные говорят, что диапазон функции один, а код выглядит иначе, перед нами уже важное аналитическое место.

Unwind-метаданные находятся между кодом и системой. Они объясняют Windows, как безопасно восстановить состояние при исключении. Поэтому ошибка в них может привести не только к красивой строке в отчете, но и к реальному изменению поведения обработки исключений.

Проект выбран удачно для обучения именно потому, что заставляет уважать служебные структуры. Реверс-инженер учится видеть не только 'что делает код', но и 'что система считает истинной картой этого кода'.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена,

которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

9. Статика и динамика как два вида доказательства

Статический анализ читает файл без запуска. Он похож на изучение чертежа: можно внимательно проверить структуру, размеры, адреса, таблицы и секции. Динамическая проверка запускает процесс и наблюдает события. Она похожа на эксперимент: можно увидеть регистрацию обработчика, исключение, точку останова, завершение процесса или тайм-аут.

Ни статика, ни динамика не являются абсолютной истиной по одиночке. Статика может показать потенциальную проблему, которая в данном запуске не проявилась. Динамика может не пройти нужную ветку, не получить нужные аргументы или остановиться по тайм-ауту. Поэтому хороший вывод всегда говорит, какой слой дал свидетельство.

В идеальном демонстрационном примере статика и динамика встречаются. Статика показывает необычное размещение, поврежденную структуру или подозрительный обработчик. Динамика показывает регистрацию таблицы, установку обработчика или срабатывание при исключении. Тогда рассказ становится сильным: файл и запуск указывают на один механизм.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

10. Что значит доказательство в учебном реверсе

В математике доказательство строится по строгим правилам. В инженерном анализе бинарника доказательство тоже должно быть дисциплинированным, но оно часто состоит из свидетельств разной силы. Есть факт байтов, факт адреса, факт секции, факт декодирования, факт события во время запуска и факт ручной проверки в Ghidra.

Слабый рассказ прыгает сразу к выводу. Сильный рассказ идет цепочкой: вот бинарник, вот команда запуска, вот отчет, вот таблица, вот адрес, вот почему он необычен, вот как это связано с документацией Windows, вот что подтвердила или не подтвердила динамика, вот что видно в Ghidra, вот ограничение вывода.

Такой стиль важен не только для защиты диплома. Это профессиональная привычка реверс-инженера: не делать вид, что инструмент знает все, а показывать, на чем держится каждое утверждение.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

11. Почему английские термины нельзя просто выкинуть

Технический мир Windows, Ghidra и официальной документации использует английские имена. Полностью убрать их нельзя: RUNTIME_FUNCTION, UNWIND_INFO, RtlAddFunctionTable и AddVectoredExceptionHandler являются официальными названиями структур и API. Если заменить их только русскими словами, потеряется связь с документацией и инструментами.

Но плохая подача оставляет эти слова как стену. Хорошая подача делает мост: сначала русский смысл, затем официальное имя, затем роль в проекте, затем способ проверки. Например, не просто 'UNWIND_INFO', а 'служебная структура Windows x64, которая описывает, как восстановить стек функции при исключении'.

Поэтому в этой книге английский термин должен быть не украшением и не списком, а адресом к точному объекту. Если термин нельзя объяснить по-русски и показать в отчете, значит он еще не стал знанием.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

12. Как из этого рождается профессия

Реверс-инженер растет не за счет количества открытых окон, а за счет точности вопросов. Что я вижу сейчас: байты, секцию, адрес, функцию, метаданные, событие запуска или итоговую оценку? Кто создал этот объект: компилятор, компоновщик, загрузчик, Windows, сама программа или анализатор? Как проверить это вторым способом?

Когда такие вопросы становятся привычкой, инструменты перестают быть магией. Visual Studio собирает бинарник, PowerShell воспроизводит команду, анализатор строит отчет, Ghidra помогает проверить адрес, JSON сохраняет точные поля, HTML помогает показать результат человеку.

Именно поэтому теория здесь не отделена от проекта. Она нужна, чтобы каждый столбец отчета, каждое поле UNWIND_INFO и каждая строка динамического лога попадали в одну картину.

Проверь себя после этого раздела: попробуй объяснить его без английских слов, а затем добавь только те официальные имена, которые действительно нужны для поиска в документации, отчете или Ghidra. Если без английского слова смысл исчезает, значит надо вернуться к русскому объяснению и укрепить связь с конкретным объектом.

Главный итог вводной теории такой: реверс-инжиниринг не начинается с героического угадывания. Он начинается с уважения к слоям. Сначала мы понимаем, где лежат байты. Потом понимаем, как Windows превращает их в образ. Потом понимаем, как функция живет в стеке. Потом понимаем, как исключение заставляет систему искать карту восстановления. Потом читаем отчет и проверяем самое важное руками. Так из хаоса терминов получается профессия.

Том I. Фундамент: от байтов к доказательству

Эта глава написана как длинная вводная лекция для человека, который раньше не занимался реверс-инжиниринг. Ее задача - не перечислить термины, а собрать картину. В центре книги находится EH/Unwind Analyzer, но понять его можно только через общую систему: файл, процессор, память, PE, загрузчик Windows, ABI, исключения, unwind метаданные, статический анализ, динамическая проверка и ручную проверку.

Читать эту главу надо медленно. Каждый раздел отвечает на вопрос "почему это вообще существует?". Почему у файла есть секции? Почему адресов несколько? Почему функции нужен стек? Почему Windows хранит unwind метаданные? Почему признак не равен verdict? Если эти вопросы проясняются, дальнейшие главы перестают быть набором сложных слов.

1. Программа как слои договоров

Когда человек без подготовки впервые слышит слово 'бинарник', оно звучит как черный ящик. На самом деле бинарник становится понятным, если смотреть на него как на цепочку договоров. Исходный код договаривается с компилятором о смысле функций и данных. Компилятор договаривается с процессором о машинных инструкциях. Компоновщик договаривается с загрузчиком Windows о формате PE. Операционная система договаривается с программой о памяти, потоках, исключениях и соглашении вызовов. Реверс-инженер входит в эту цепочку с обратной стороны и восстанавливает, какие договоры видны в готовом файле.

EH/Unwind Analyzer занимается не всей программой сразу. Его область уже и точнее: он изучает то место, где PE, Windows x64 ABI и механизм исключений встречаются в таблица функций времени выполнения и unwind метаданные. Это узкая тема, но она глубоко

связана с фундаментом. Нельзя понять UNWIND_INFO, если не понимать стек. Нельзя понять стек, если не понимать функцию. Нельзя понять функцию, если не понимать соглашение вызовов. Нельзя понять адрес обработчик, если путать RVA и VA.

Поэтому правильное чтение отчета начинается не с verdict. Verdict - это конец маршрута. Начало маршрута - вопрос: какой договор сейчас проверяется? Если проверяется PE layout, мы говорим о секциях, размерах, raw offset и RVA. Если проверяется function table, мы говорим о begin, end и unwind RVA. Если проверяется динамика, мы говорим о реальном запуске, события отладки, API calls и срабатывания обработчиков. Глубина появляется тогда, когда эти слои не смешиваются.

Проверка понимания простая: попробуй объяснить один и тот же адрес тремя способами. В файле это смещение от начала байтов. В PE image это RVA относительно ImageBase. В запущенном процессе это VA. Если эти три объяснения не путаются, значит ты уже вышел из режима 'черный ящик' и начал думать как реверс-инженер.

Короткая самопроверка: сформулируй этот раздел одним предложением, затем найди в отчете проекта поле или таблицу, где эта идея становится видимой.

2. Почему байты не равны смыслу

Байт сам по себе ничего не говорит. Один и тот же байт может быть частью инструкции, числом, символом строки, полем структуры, padding или случайным мусором. Смысл появляется только из контекста. В PE этот контекст задают заголовки и секции. В коде этот контекст задают инструкции и поток управления. В unwind метаданные этот контекст задают правила Windows x64 обработка исключений.

Например, число 0x1000 может быть RVA начала секции, размером, константой в коде или частью машинной инструкции. Если аналитик видит число и сразу делает вывод, он гадает. Если аналитик спрашивает, кто владеет этим числом и в какой структуре оно

находится, он анализирует. Именно поэтому проект сначала строит модель PE, а только потом читает exception directory и функции времени выполнения.

Через эту мысль становится понятной роль парсер. Парсер не 'ищет вирус'. Парсер делает более скромную и важную работу: он превращает сырые байты в проверяемые структуры, не доверяя входу. Если размер секции выходит за файл, парсер должен заметить. Если RVA нельзя перевести в файловое смещение, декодер не имеет права читать дальше. Это не бюрократия, а безопасность анализа.

Практический навык: когда видишь любое поле в отчете, не называй его сразу 'подозрительным'. Сначала назови его тип: байты, offset, RVA, VA, размер, флаг, имя секции, адрес обработчик, событие dynamic log. Потом скажи, из какой структуры оно пришло. Только после этого можно обсуждать нормальность или аномальность.

Короткая самопроверка: сформулируй этот раздел одним предложением, затем найди в отчете проекта поле или таблицу, где эта идея становится видимой.

3. Функция, стек и причина существования unwind метаданные

Функция в исходном коде кажется простой: есть имя, аргументы, тело и return. В бинарнике функция становится соглашением о переходе управления и состоянии процессора. Инструкция call кладет адрес возврата, функция может выделить место на стеке, сохранить регистры, вызвать другие функции и затем вернуть управление. Все это должно быть сделано так, чтобы вызывающая и вызываемая стороны не разрушили состояние друг друга.

Стек нужен как рабочая память вызовов. Там лежат адреса возврата, локальные данные, сохраненные регистры и иногда аргументы. Но стек не является самодокументирующимся. Если исключение происходит внутри функции, Windows должна понять, как

вернуться к предыдущему кадру. Она не может просто 'посмотреть глазами'. Ей нужна формальная карта того, что функция успела сделать со стеком и регистрами.

Эта карта и есть unwind метаданные. UNWIND_INFO описывает prolog, размер стека, сохранения регистров и наличие обработчика. RUNTIME_FUNCTION говорит, для какого диапазона кода эта карта действует. Поэтому таблица функций времени выполнения - это не второстепенная таблица. Это инфраструктура, без которой корректная обработка исключений и stack unwinding на x64 становятся ненадежными.

Отсюда понятно, почему аномалии в этой области важны. Если unwind RVA указывает за пределы образа, карта не читается. Если метаданные лежит в writable секция, возникает вопрос, можно ли ее менять во время выполнения. Если обработчик указывает в non-executable область, это противоречит ожиданию о коде обработчика. Это не магические признаки, а нарушения вокруг механизма восстановления управления.

Короткая самопроверка: сформулируй этот раздел одним предложением, затем найди в отчете проекта поле или таблицу, где эта идея становится видимой.

4. PE как карта между диском и памятью

Файл на диске и программа в памяти не совпадают один к одному. PE описывает, как Windows должна превратить файл в image: какие секции загрузить, куда они попадут, какие права получат, где лежат импорты, релокации, ресурсы и exception directory. Поэтому PE - это карта перехода между двумя мирами: raw file и virtual memory.

Секция имеет raw offset и raw size для файла, а также virtual address и virtual size для памяти. Эти пары легко перепутать. Если аналитик открывает hex viewer, он находится в мире file offsets. Если он открывает Ghidra с ImageBase, он чаще всего находится в мире virtual addresses. Если он читает RUNTIME_FUNCTION, он работает с RVA.

Один и тот же объект надо уметь переносить между этими мирами.

EH/Unwind Analyzer делает такой перенос постоянно. Когда он видит UnwindInfoAddress, он должен понять, какая секция содержит этот RVA, можно ли отобразить его в файловое смещение, достаточно ли байтов осталось до конца области, какие права у секции и как это соотносится с ожиданиями Windows. Если на любом шаге возникает противоречие, появляется конкретный признак.

Глубокое понимание PE начинается с простой дисциплины: никогда не говорить 'адрес' без уточнения. Говори файловое смещение, RVA или VA. Говори секция и range. Говори raw size или virtual size. Тогда отчет перестает быть набором чисел и превращается в проверяемую карту.

Короткая самопроверка: сформулируй этот раздел одним предложением, затем найди в отчете проекта поле или таблицу, где эта идея становится видимой.

5. Статический анализ как чтение договора

Статический анализ не запускает программу. Это не недостаток, а особенность метода. Он читает то, что записано в файле: заголовки, секции, таблицы, метаданные, строки, импорты, диапазоны функций. Его сила в том, что он видит структуру целиком и не зависит от того, прошла ли программа нужную ветку во время запуска.

Но статический анализ не должен выдавать себя за поведенческое доказательство. Если метаданные необычна, это факт файла. Если обработчик лежит вне ожидаемой секции, это факт размещения. Если время выполнения table не отсортирована, это факт структуры. Эти факты сильны, но они не равны утверждению 'при запуске обязательно произойдет такое-то действие'.

Проект поэтому разделяет static признаки и динамическая проверка. Static признаки говорят: в PE есть проверяемое нарушение или необычность. Динамическая проверка говорит: во время

конкретного запуска были такие-то события. Когда эти два слоя поддерживают друг друга, вывод становится сильнее. Когда динамика молчит, статический факт не исчезает, но его надо формулировать осторожнее.

Хорошая фраза для защиты: 'Статически я вижу нарушение такого-то инварианта. Это не приговор, а структурный факт. Дальше я проверяю, есть ли время выполнения-события, которые усиливают вывод, и показываю ручную проверку в Ghidra или JSON'.

Короткая самопроверка: сформулируй этот раздел одним предложением, затем найди в отчете проекта поле или таблицу, где эта идея становится видимой.

6. Динамическая проверка как эксперимент

Динамика кажется убедительной, потому что программа реально запускается. Но любой запуск - это эксперимент с условиями. Есть аргументы командной строки, текущая папка, права, загруженные DLL, тайм-аут, путь выполнения, случайность и состояние системы. Один запуск не покрывает всю программу. Он показывает только то, что произошло в данном сценарии.

Debug API Windows позволяет наблюдать события процесса: создание процесса, загрузку DLL, исключения, завершение, точки останова. EH/Unwind Analyzer использует это, чтобы увидеть регистрацию обработчиков, вызовы время выполнения table API и попадания в обработчик. Это ценно, потому что такие события уже не просто лежат в файле, а происходят во время выполнения.

Но динамический отчет надо читать вместе с его ограничениями. Тайм-аут reached означает, что эксперимент остановлен по времени. Точка останова failures говорят, что часть точек наблюдения могла не установиться. Потерянные события говорят, что collector мог не сохранить все события. Полнота инструментирования показывает, насколько наблюдение завершено. Эти поля не мелочь, а научная честность.

Поэтому правильный вывод звучит так: 'В этом запуске наблюдались такие-то события'. Неправильный вывод звучит так: 'В программе больше ничего нет'. Реверс-инженер силен не громкостью утверждения, а точностью границ.

Короткая самопроверка: сформулируй этот раздел одним предложением, затем найди в отчете проекта поле или таблицу, где эта идея становится видимой.

7. Ghidra как место ручной проверки

Ghidra не является волшебным судьей. Это мощная среда реверс-инжиниринг: дизассемблерное представление, decompiler, карта памяти, functions, ссылки, scripts. Она помогает человеку увидеть код и данные, но ее результат тоже является анализом, а не исходной истиной. Разметка функции может отличаться от RUNTIME_FUNCTION. Decompiler может упростить или неверно восстановить намерение. Поэтому Ghidra надо использовать с вопросом.

В этом проекте главный вопрос к Ghidra обычно такой: где находится representative RVA из отчета и что вокруг него видно? Если отчет говорит о обработчик, нужно перейти к адресу обработчик. Если отчет говорит об unwind метаданные, нужно посмотреть секцию и связанные диапазоны. Если отчет говорит о странной секции, нужно открыть карта памяти и проверить права.

Правильный workflow: сначала отчет формулирует вопрос, затем Ghidra помогает проверить контекст. Неправильный workflow: открыть весь бинарник и бесцельно искать 'что-то страшное'. Ручная проверка должна быть адресной. У нее есть вход: RVA, секция, признак code, обработчик или время выполнения event. У нее есть выход: подтверждение, ослабление или уточнение вывода.

На защите это выглядит сильно: 'Вот строка отчета. Вот RVA. Вот расчет адреса для Ghidra. Вот окружающий дизассемблерное представление или карта памяти. Вот почему это подтверждает или

уточняет признак'. Такой показ превращает инструмент из черного ящика в проверяемую систему.

Короткая самопроверка: сформулируй этот раздел одним предложением, затем найди в отчете проекта поле или таблицу, где эта идея становится видимой.

8. От признака к доказательству

Признак - это не обвинение. Признак - это наблюдение, которому дали имя. Например, `unwind_parse_fail` говорит, что UNWIND_INFO не удалось корректно разобрать. `rf_unsorted` говорит, что порядок время выполнения function entries нарушает ожидание. `handler_nonexec_section` говорит, что обработчик связан с областью без executable permission. В каждом случае важно не имя признака, а цепочка доказательства.

Цепочка доказательства состоит из нескольких звеньев. Первое - объект: какая функция, секция, таблица или обработчик. Второе - координата: RVA, VA или файловое смещение. Третье - правило: какой инвариант Windows/PE/x64 ожидается. Четвертое - наблюдение: что именно нарушено. Пятое - проверка: где это видно в HTML, JSON, Ghidra или журнал выполнения. Шестое - ограничение: чего этот факт не доказывает.

Если хотя бы одно звено пропущено, вывод становится слабым. Если нет координаты, нечего проверять. Если нет правила, непонятно, почему это аномалия. Если нет ограничения, вывод звучит как преувеличение. Поэтому хороший отчет должен не только ставить оценка, но и показывать details, affected count, representative RVA, Функции времени выполнения и Manual validation plan.

Финальный навык реверс-инженера - говорить спокойно. Не 'это точно вредонос'. А 'это необычное размещение unwind метаданные; оно затрагивает такие-то функции; динамический запуск показал такие-то события; ручная проверка должна подтвердить такие-то адреса; уверенность ограничена такими-то условиями'. Это и есть

инженерная глубина.

Короткая самопроверка: сформулируй этот раздел одним предложением, затем найди в отчете проекта поле или таблицу, где эта идея становится видимой.

Итог фундаментальной главы

После этой главы у тебя должна появиться главная мысль: реверс-инжиниринг не является гаданием по непонятным байтам. Это восстановление договоров. PE договаривается с Windows о загрузке. ABI договаривается о вызовах функций. Unwind метаданные договаривается о восстановлении стека. Analyzer проверяет, где эти договоры выполнены, нарушены или выглядят необычно. Человек затем проверяет самые важные места руками.

Если держать эту рамку, даже сложные слова становятся рабочими. RVA - это не страшная аббревиатура, а координата внутри image. RUNTIME_FUNCTION - не просто строка таблицы, а связь диапазона кода с картой восстановления. UNWIND_INFO - не мусорная структура, а инструкция для раскрутки стека. Dynamic log - не шум, а дневник эксперимента. Ghidra - не оракул, а микроскоп для проверки адресного вопроса.

Том I-A. Windows x64 EH/Unwind: как механизм работает изнутри

Этот раздел углубляет центральную тему проекта. Если фундаментальная глава объясняет общую картину, то здесь мы медленно разбираем именно EH/Unwind: исключение, контекст потока, поиск функции, UNWIND_INFO, обработчик, динамические таблицы и итоговую оценку. Цель - чтобы слова из отчета перестали быть ярлыками и стали понятными частями одного механизма.

1. Почему исключение - это не просто ошибка

В бытовом смысле исключение часто воспринимается как 'что-то сломалось'. В системном смысле exception - это механизм передачи управления при особом событии. Событие может быть ошибкой доступа к памяти, делением на ноль, точка останова, single step, программно сгенерированным исключением или ситуацией, которую время выполнения использует как часть своего поведения.

Для реверс-инженера важно не путать событие и смысл. Сам факт exception не доказывает вредоносность. Отладчик тоже использует exceptions. Защитные механизмы и антиотладка тоже используют exceptions. Обычный время выполнения может использовать обработку исключений легально. Поэтому анализатор считает не эмоцию, а структуру: где зарегистрирован обработчик, какая таблица функций существует, какой код был достигнут при запуске.

Windows должна решить, кому передать управление после exception. Для этого она использует контекст потока, адрес инструкции, таблицы функций, обработчики и правила поиска. Если программа x64 имеет функции с изменением стека, система должна уметь правильно подняться по кадрам. Поэтому исключение сразу связывает CPU context, stack, ABI и unwind метаданные.

В отчете это проявляется двумя путями. Статически анализатор читает таблицы, которые Windows будет использовать. Динамически

анализатор наблюдает события, которые реально произошли. Глубина анализа появляется тогда, когда ты понимаешь, какая часть относится к потенциальной карте поведения, а какая - к фактическому запуску.

Проверка на практике: найди в любом отчете проекта поле, которое соответствует этой теме, и объясни его без слова 'просто'. Если объяснение требует адрес, покажи RVA и секцию.

2. Контекст потока и точка остановки мира

Когда exception происходит, поток имеет конкретное состояние: регистры, RSP, RIP, flags, память стека. Это состояние называют context. Оно важно потому, что обработчик должен понять, где выполнение остановилось и как можно продолжить, завершить или передать управление дальше.

На x64 адрес текущей инструкции и состояние стека определяют, какую RUNTIME_FUNCTION нужно искать. Если RIP находится внутри диапазона BeginAddress-EndAddress, Windows получает соответствующий UnwindInfoAddress. Без этой связи невозможно аккуратно понять, как восстановить предыдущий frame.

Отладчик видит exception как debug event. Это не то же самое, что обычный printf в программе. Потоки процесса приостанавливаются, отладчик получает событие, решает, что делать, и затем продолжает процесс. Поэтому dynamic collector работает в мире событий и синхронизации, а не просто читает лог программы.

В отчете Журнал выполнения каждая строка про exception, точка останова или срабатывание обработчика связана с этим механизмом. Если ты читаешь лог как обычный текст, он кажется шумом. Если читаешь его как историю остановок и продолжений потока, он становится понятным.

Проверка на практике: найди в любом отчете проекта поле, которое соответствует этой теме, и объясни его без слова 'просто'. Если

объяснение требует адрес, покажи RVA и секцию.

3. Поиск функции по RIP

RUNTIME_FUNCTION table нужна, чтобы по адресу инструкции найти описание функции. Представь библиотечный каталог: если ты знаешь номер страницы, тебе нужно понять, в какой главе эта страница находится. BeginAddress и EndAddress задают диапазон. UnwindInfoAddress говорит, где лежит инструкция по восстановлению.

Если таблица отсортирована, поиск может быть быстрым и однозначным. Если диапазоны перекрываются, один RIP может попадать сразу в несколько описаний. Если begin больше или равен end, диапазон не имеет смысла. Если UnwindInfoAddress не отображается в файл, статический анализатор не может безопасно прочитать карту.

Эти нарушения не являются декоративными. Они касаются самого механизма поиска. Поэтому признаки вроде rf_unsorted, rf_overlap, rf_bad_range и unwind_rva_oob имеют смысл не как набор ругательств, а как проверка пригодности таблицы для надежного объяснения кода.

Ручная проверка в Ghidra здесь проста: возьми BeginAddress, прибавь ImageBase, перейди к адресу и посмотри, есть ли там код. Затем сравни границы функции Ghidra с EndAddress. Если они расходятся, объясни источник каждого взгляда.

Проверка на практике: найди в любом отчете проекта поле, которое соответствует этой теме, и объясни его без слова 'просто'. Если объяснение требует адрес, покажи RVA и секцию.

4. Что именно хранит UNWIND_INFO

UNWIND_INFO можно представить как компактную инструкцию для обратного движения по функции. Она не хранит исходный код. Она хранит сведения, важные для восстановления состояния: версию, флаги, размер prolog, количество unwind codes, описание операций со стеком и регистрами, а иногда обработчик или chained info.

Prolog важен потому, что именно там функция обычно меняет RSP и сохраняет nonvolatile registers. Если exception случился после prolog, Windows должна знать, как отменить эти действия. Если exception случился внутри prolog, правила становятся еще аккуратнее. Поэтому unwind codes имеют порядок и смысл, а не являются произвольным массивом.

Flags меняют хвост структуры. EHANDLER или UHANDLER означают, что после unwind codes есть адрес обработчика. CHAININFO означает, что структура ссылается на другую RUNTIME_FUNCTION. Если парсер не учитывает flags, он будет читать не те байты как не те поля. Поэтому декодер должен быть строгим.

В проекте parse failure важен именно здесь. Он говорит: по указанному адресу структура не была прочитана как корректная UNWIND_INFO по ожидаемым правилам. Это не 'не понравилось', а конкретный сбой декодирования.

Проверка на практике: найди в любом отчете проекта поле, которое соответствует этой теме, и объясни его без слова 'просто'. Если объяснение требует адрес, покажи RVA и секцию.

5. Обработчик как код, а не просто число

Обработчик RVA в отчете - это не абстрактный номер. Это адрес кода или обратный вызов, которому система может передать управление в процессе обработки exception. Поэтому вокруг обработчик появляются вопросы: попадает ли он в image, в какую секцию попадает, исполняемая ли эта секция, не доминирует ли один обработчик над большим числом функций, не выглядит ли reuse слишком концентрированным.

Если обработчик указывает за пределы image, это серьезный структурный вопрос. Если обработчик находится в non-executable секция, возникает конфликт с ожиданием исполняемого кода. Если обработчик находится в writable секция, возникает вопрос

изменяемости. Если один обработчик обслуживает слишком много функций, это может быть легальным время выполнения-паттерном, но также может быть признаком обфускации или необычной схемы.

Динамическая проверка усиливает этот слой, когда обработчик реально достигается. Обработчик hit говорит, что точка останова на обработчик сработал. Но даже тут надо смотреть контекст: какое exception было, first chance или second chance, какой поток, какой адрес, завершился ли процесс нормально или аварийно.

На защите обработчик надо показывать не словами, а маршрутом: обработчик RVA в отчете, секция и permissions, переход в Ghidra, строка Журнал выполнения при динамическом подтверждении. Тогда число становится доказательством.

Проверка на практике: найди в любом отчете проекта поле, которое соответствует этой теме, и объясни его без слова 'просто'. Если объяснение требует адрес, покажи RVA и секцию.

6. Динамические таблицы функций

Не все функции обязаны быть описаны только статической .pdata в файле. Windows поддерживает динамические таблицы функций через API вроде RtlAddFunctionTable и RtlInstallFunctionTableCallback. Это нужно, например, для JIT, время выполнения-generated code и сложных сред выполнения. Но для анализа это важный время выполнения-сигнал.

Если процесс вызывает RtlAddFunctionTable, он сообщает Windows о таблице функций во время выполнения. Если вызывает RtlInstallFunctionTableCallback, он может устанавливать обратный вызов для получения function table information. Это легально, но аналитически интересно, потому что карта unwind поведения становится частично динамической.

Статический анализ может не увидеть всю такую карту заранее. Он видит файл. Динамический collector может увидеть сам факт

регистрации. Поэтому Время выполнения table calls в отчете имеет особый вес: это не просто странное поле PE, а время выполнения-действие вокруг механизма unwind.

Правильный вывод не должен преувеличивать: вызов RtlAddFunctionTable не делает программу вредоносной. Но если он совпадает с необычным static placement, обработчик activity или обфускационной структурой, корреляция становится сильной.

Проверка на практике: найди в любом отчете проекта поле, которое соответствует этой теме, и объясни его без слова 'просто'. Если объяснение требует адрес, покажи RVA и секцию.

7. Почему writable unwind метаданные настораживает

Метаданные обычно воспринимается как описание, а не как исполняемое поведение. Но в системах низкого уровня описание может управлять поведением. Если Windows использует метаданные для восстановления стека и поиска обработчик, то изменение метаданные способно менять то, как система понимает выполнение.

Writable секция означает, что область может быть изменена во время выполнения. Это не всегда зло: бывают время выполнения-паттерны, packer, loader, JIT, самописные схемы. Но для unwind метаданные writable placement необычен, потому что карта восстановления стека обычно должна быть стабильной частью образа.

Поэтому признак `unwind_section_writable` надо объяснять осторожно. Он не говорит 'это malware'. Он говорит: 'данные, от которых зависит unwind/exception interpretation, находятся в изменяемой области'. Дальше нужен контекст: кто пишет в эту область, есть ли dynamic table calls, есть ли срабатывания обработчиков, есть ли признаки обфускации.

Ручная проверка: открыть Sections, найти секцию Unwind RVA, проверить permissions, затем открыть Функции времени выполнения и посмотреть, сколько функций ссылаются на эту область. Если много функций, признак затрагивает не одну строку, а общий участок метаданные.

Проверка на практике: найди в любом отчете проекта поле, которое соответствует этой теме, и объясни его без слова 'просто'. Если объяснение требует адрес, покажи RVA и секцию.

8. Как читать итоговую оценку без самообмана

Final assessment удобен, но опасен, если читать только его. Verdict, уверенность, static/свидетельства времени выполнения и cross-layer correlation являются сжатием большого анализа. Сжатие помогает обзорно, но не заменяет доказательство.

Если final уверенность высокое, спроси: за счет чего? Есть ли high severity признаки? Есть ли dynamic signals? Есть ли срабатывания обработчиков или время выполнения table calls? Есть ли manual validation plan? Если уверенность среднее, спроси, чего не хватает: динамического подтверждения, полной instrumentation, большего числа независимых признаков или ручной проверки.

Хороший инженер не прячет uncertainty. Он показывает, почему вывод имеет именно такую силу. Если есть только статика, он говорит 'структурно подозрительно'. Если есть статика и динамика, он говорит 'структурно необычно и время выполнения-подтверждено в таком-то сценарии'. Если есть тайм-аут, он не делает вывод о полном поведении.

Так отчет становится учебным инструментом. Он не заменяет мышление, а заставляет пройти правильный маршрут: факт, адрес, правило, проверка, ограничение, вывод.

Проверка на практике: найди в любом отчете проекта поле, которое соответствует этой теме, и объясни его без слова 'просто'. Если

объяснение требует адрес, покажи RVA и секцию.

Том I-B. Типичные ошибки новичка: как не разрушить сильный анализ

Эта глава нужна потому, что глубокое понимание проявляется не только в знании терминов, но и в умении не делать ложных скачков. Реверс-инженер постоянно удерживает границы вывода: где факт, где интерпретация, где подтверждение, где ограничение эксперимента. Ошибки ниже встречаются часто, и каждая из них может испортить даже хороший отчет.

На защите эти ошибки обычно проявляются в форме простых вопросов. "Почему вы называете это аномалией?" "Где этот адрес в Ghidra?" "Почему динамика ничего не показала?" "Почему affected count такой большой?" "Что означает уверенность?" Если отвечать на такие вопросы общими словами, работа кажется сырой. Если отвечать через слой, адрес, правило и проверку, даже сложная тема становится убедительной.

Поэтому ниже каждая ошибка дана в двух формах: как выглядит слабое рассуждение и как его заменить сильным. Не надо заучивать формулировки дословно. Надо усвоить ход: остановиться, назвать объект, уточнить координату, объяснить правило, показать проверку и только потом делать вывод.

Ошибка 1. Путать странность с вредоносностью

Необычная секция, writable метаданные или обработчик reuse не равны слову malware. Это признаки, которые требуют проверки. Если сразу назвать их вредоносностью, защита станет слабее: достаточно одного легального примера JIT, packer или нестандартного время выполнения, чтобы формулировка выглядела грубой.

Правильная фраза: 'обнаружен структурный признак, который необычен для ожидаемой схемы Windows x64 unwind метаданные; он повышает подозрительность и требует ручной проверки'. Такая фраза точнее и профессиональнее.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 2. Смешивать координаты

Самая частая техническая ошибка - говорить 'адрес' без уточнения. Файловое смещение, RVA и VA живут в разных слоях. Отчет может дать RVA, Ghidra показать VA, hex viewer открыть файловое смещение. Если не сделать перевод, можно показать преподавателю не то место.

Правильная дисциплина: рядом с каждым числом пиши его тип. `0x7100` RVA, файловое смещение `0x900`, VA `0x140007100`. Тогда даже сложный отчет становится проверяемым.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 3. Верить decompiler вместо механизма

Decompiler удобен, но он восстанавливает C-подобное представление. Для EH/Unwind темы иногда важнее дизассемблерное представление, prolog, stack adjustment, секция permissions и метаданные. Если decompiler молчит про обработчик, это не значит, что обработчик метаданные не существует.

Правильная проверка: сначала смотри таблицу Функции времени выполнения и UNWIND_INFO, затем дизассемблерное представление вокруг адреса, и только потом decompiler как удобное объяснение логики.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 4. Игнорировать нормальные образцы

Если показывать только подозрительные бинарники, создается впечатление, что инструмент всегда находит тревогу. Нормальный benign образец нужен как контроль. Он показывает, как выглядит спокойная таблица, OK-статусы и отсутствие strong anomalies.

Правильная демонстрация: сначала коротко показать benign report, затем перейти к комбинированный образец. Контраст делает подозрительный пример понятнее.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 5. Считать тайм-аут отрицательным доказательством

Если динамика завершилась тайм-аут, это не значит, что события отсутствуют. Это значит, что в данном временном окне они не были достигнуты или процесс не завершился. Тайм-аут ограничивает вывод, а не отменяет статические факты.

Правильная формулировка: 'динамический запуск не дал подтверждения в выбранных условиях; статический признак остается структурным наблюдением'.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 6. Читать только верхнюю карточку отчета

Final assessment помогает быстро сориентироваться, но не содержит всего доказательства. Настоящая проверка находится ниже: Static признаки, Функции времени выполнения, Журнал выполнения, Analysis details, Manual validation plan.

Правильный порядок: сначала карточка, потом конкретный признак, потом affected object и representative RVA, потом ручная проверка.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 7. Не отличать парсер failure от плохой программы

Parse failure говорит о том, что анализатор не смог корректно прочитать структуру по ожидаемым правилам. Причина может быть в повреждении, обфускации, неподдержанном варианте, нехватке байтов или ошибке инструмента. Это не автоматический verdict.

Правильная проверка: открыть bytes по RVA, проверить границы секции, version/flags/count fields и сравнить с документацией Microsoft по x64 обработка исключений.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 8. Защищать вывод без команды воспроизведения

Отчет без команды запуска похож на скриншот без эксперимента. Для диплома важно показать воспроизводимость: какой анализатор, какой образец, какие опции, какой тайм-аут, какие выходные файлы.

Правильная привычка: рядом с каждым показательным HTML держать PowerShell-команду, которая пересоздает JSON/HTML и при необходимости динамический отчет.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 9. Ждать аномалию в каждой строке Функции времени выполнения

Если общий признак относится к секции или общей области метаданные, он может затрагивать десятки функций, но не обязан превращать каждую строку Функции времени выполнения в уникальную ошибку. Строка может быть OK как декодированная структура, но при этом ссылаться на секцию, которая в целом необычна.

Правильное чтение: смотри и верхние Static признаки, и колонку Anomalies/error, и секция summary. Если аномалия общая для секции, объясняй ее как свойство размещения, а не как отдельную поломку каждой функции.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 10. Путать affected count с числом разных причин

Affected count показывает, сколько объектов затронуто признаком. Это не обязательно число разных аномалий. Один плохой UnwindInfoRVA может затронуть одну функцию, а одна writable unwind секция может затронуть много функций сразу.

Правильный ответ: сначала назови признак code, затем affected count, затем объясни, является ли причина локальной для одной строки или общей для секции/таблицы.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 11. Считать уверенность процентом истины

Уверенность в отчете - инженерная оценка силы свидетельства, а не математическая вероятность того, что файл вредоносный. Если написано 69%, это не значит 'на 69 процентов malware'. Это значит, что совокупность признаков имеет такую расчетную силу в модели проекта.

Правильная формулировка: 'уверенность отражает силу и согласованность найденных свидетельств; окончательный вывод требует ручной проверки и контекста образец'.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 12. Не сверять HTML с JSON

HTML удобен, но при споре лучше иметь JSON. В JSON поля не спрятаны верстkой: признак code, severity, affected, representative RVA, dynamic события и counts можно проверить точно. Если защита требует строгости, JSON сильнее скриншота.

Правильная проверка: для одного признак найди его в HTML, затем найди соответствующий объект в JSON и сравни code, message, RVA и affected count.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Ошибка 13. Готовить GitHub и презентацию без чистоты артефактов

Даже хороший проект выглядит слабее, если рядом лежат временные PNG, старые candidates, мусорные build-cache файлы и случайные отчеты без назначения. Для презентации и GitHub нужна чистая структура: исходники, тесты, docs, reports-примеры, scripts и понятный README.

Правильная привычка: перед загрузкой проверить временные папки, убрать промежуточные PDF-render PNG, оставить только воспроизводимые документы и примеры, которые ты реально можешь объяснить.

Мини-практика: открой один отчет и найди место, где эту ошибку легко совершить. Затем сформулируй корректный вариант ответа.

Контрольные вопросы преподавателя

Вопрос: 'Почему вы уверены, что это именно unwind-анализ, а не общий антивирусный сканер?' Ответ: потому что объект анализа ограничен конкретными структурами Windows x64: PE exception directory, RUNTIME_FUNCTION, UNWIND_INFO, обработчик RVA, секция placement и время выполнения exception/table события. Инструмент не ищет сигнатуры malware. Он проверяет договоры вокруг EH/Unwind.

Вопрос: 'Почему статический признак важен, если динамика его не подтвердила?' Ответ: статический признак является фактом структуры файла. Динамика проверяет только конкретный путь выполнения при конкретных условиях. Отсутствие время выполнения-события ослабляет поведенческий вывод, но не удаляет структурное наблюдение.

Вопрос: 'Почему один обработчик на много функций подозрителен?' Ответ: сам по себе reuse может быть легальным. Подозрительность появляется, когда концентрация обработчик сочетается с необычным placement, writable метаданные, странной секцией или dynamic behavior. Поэтому важно говорить не о количестве само по себе, а о совокупности свидетельства.

Вопрос: 'Что вы делаете, если Ghidra показывает не те границы функции?' Ответ: я разделяю источники. Analyzer читает PE время выполнения function метаданные. Ghidra строит свою function model по дизассемблерное представление и эвристикам. Я сравниваю оба взгляда, проверяю bytes, секция, prolog и ссылки, а не выбираю один

инструмент как абсолютную истину.

Вопрос: 'Как доказать воспроизводимость?' Ответ: показать команду запуска, input образец, версию анализатор, опции static/dynamic, тайм-аут, пути HTML/JSON/CSV и затем заново получить тот же тип отчета. Для динамики я дополнительно называю ограничения запуска.

Вопрос: 'Где здесь место человека?' Ответ: анализатор поднимает структурные и время выполнения-факты, но человек связывает их с контекстом, проверяет адреса в Ghidra, читает JSON, сравнивает benign и suspicious образцы и формулирует ограниченный вывод. Поэтому проект является инструментом поддержки реверс-инжиниринг, а не заменой эксперта.

Том II-A. Англоязычные термины: как переводить, понимать и объяснять

В реверс-инжиниринге невозможно полностью убрать английские слова: документация Microsoft, Ghidra, имена структур и API написаны по-английски. Но плохой учебник оставляет их как список. Хороший учебник делает другое: рядом с английским термином дает русский смысл, техническую роль, типичную ошибку и способ проверки.

Эта глава нужна как мост. Ее задача - чтобы слова время выполнения, обработчик, метаданные, признак, уверенность, секция, RVA не висели в воздухе. После каждого термина ты должен уметь сказать: что это значит по-русски, где это видно в проекте, почему это важно и как это проверить.

1. byte: байт

Переводить byte надо не одним словом, а смыслом. В этой книге рабочий русский вариант - байт. Коротко: минимальная единица данных в файле. Байт сам по себе не имеет смысла, пока мы не знаем, является ли он частью инструкции, числа, строки, поля PE или служебного выравнивания.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин byte я использую не как непереведенное слово, а как название конкретного объекта. По-русски это байт; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

2. файловое смещение: файловое смещение

Переводить файловое смещение надо не одним словом, а смыслом. В этой книге рабочий русский вариант - файловое смещение. Коротко: позиция байта от начала файла на диске. Это координата hex viewer. Она нужна, когда мы доказываем, что структура действительно читается из файла.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин файловое смещение я использую не как непереведенное слово, а как название конкретного объекта. По-русски это файловое смещение; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

3. RVA: относительный виртуальный адрес

Переводить RVA надо не одним словом, а смыслом. В этой книге рабочий русский вариант - относительный виртуальный адрес. Коротко: адрес внутри загруженного образа относительно ImageBase. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин RVA я использую не как непереведенное слово, а как название конкретного объекта. По-русски это относительный виртуальный адрес; в проекте это проверяется через отчет, JSON и ручной переход к адресу или

событию'.

4. VA: виртуальный адрес

Переводить VA надо не одним словом, а смыслом. В этой книге рабочий русский вариант - виртуальный адрес. Коротко: адрес в памяти конкретного процесса. VA получается из ImageBase и RVA; именно такие адреса часто видны в отладчик и Ghidra.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин VA я использую не как непереведенное слово, а как название конкретного объекта. По-русски это виртуальный адрес; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

5. PE: Portable Executable

Переводить PE надо не одним словом, а смыслом. В этой книге рабочий русский вариант - Portable Executable. Коротко: формат исполняемых файлов Windows. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин PE я использую не как непереведенное слово, а как название конкретного объекта. По-русски это Portable Executable; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

6. секция: секция PE

Переводить секция надо не одним словом, а смыслом. В этой книге рабочий русский вариант - секция PE. Коротко: именованный диапазон файла и памяти с правами доступа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин секция я использую не как непереведенное слово, а как название конкретного объекта. По-русски это секция PE; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

7. метаданные: метаданные

Переводить метаданные надо не одним словом, а смыслом. В этой книге рабочий русский вариант - метаданные. Коротко: данные, описывающие другие данные или код. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин метаданные я использую не как непереведенное слово, а как название конкретного объекта. По-русски это метаданные; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

8. ABI: двоичный интерфейс приложения

Переводить ABI надо не одним словом, а смыслом. В этой книге рабочий русский вариант - двоичный интерфейс приложения. Коротко: правила вызова функций, регистров, стека и выравнивания. Без ABI нельзя понять prolog, epilog, кадр стека и смысл UNWIND_INFO.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин ABI я использую не как непереведенное слово, а как название конкретного объекта. По-русски это двоичный интерфейс приложения; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

9. stack: стек

Переводить stack надо не одним словом, а смыслом. В этой книге рабочий русский вариант - стек. Коротко: память потока для вызовов функций и временного состояния. Стек связывает call/return, локальные данные, сохраненные регистры и обработку исключений.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин stack я использую не как непереведенное слово, а как название конкретного объекта. По-русски это стек; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

10. frame: кадр стека

Переводить `frame` надо не одним словом, а смыслом. В этой книге рабочий русский вариант - кадр стека. Коротко: участок стека, принадлежащий конкретной функции. Unwind метаданные нужна, чтобы восстановить предыдущий frame при исключении.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин `frame` я использую не как непереведенное слово, а как название конкретного объекта. По-русски это кадр стека; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

11. RUNTIME_FUNCTION: запись таблицы функций времени выполнения

Переводить `RUNTIME_FUNCTION` надо не одним словом, а смыслом. В этой книге рабочий русский вариант - запись таблицы функций времени выполнения. Коротко: тройка BeginAddress, EndAddress и UnwindInfoAddress. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин `RUNTIME_FUNCTION` я использую не как непереведенное слово, а как название конкретного объекта. По-русски это запись таблицы функций времени выполнения; в проекте это проверяется через отчет, JSON и ручной переход к

адресу или событию'.

12. UNWIND_INFO: информация раскрутки стека

Переводить UNWIND_INFO надо не одним словом, а смыслом. В этой книге рабочий русский вариант - информация раскрутки стека. Коротко: структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Она объясняет системе, как безопасно вернуться к вызывающему кадру.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин UNWIND_INFO я использую не как непереведенное слово, а как название конкретного объекта. По-русски это информация раскрутки стека; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

13. обработчик: обработчик исключения

Переводить обработчик надо не одним словом, а смыслом. В этой книге рабочий русский вариант - обработчик исключения. Коротко: код или обратный вызов, которому может быть передано управление. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин обработчик я использую не как непереведенное слово, а как название конкретного объекта.

По-русски это обработчик исключения; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

14. статический анализ: статический анализ

Переводить статический анализ надо не одним словом, а смыслом. В этой книге рабочий русский вариант - статический анализ. Коротко: чтение файла без запуска. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин статический анализ я использую не как непереведенное слово, а как название конкретного объекта. По-русски это статический анализ; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

15. динамическая проверка: динамическая проверка

Переводить динамическая проверка надо не одним словом, а смыслом. В этой книге рабочий русский вариант - динамическая проверка. Коротко: наблюдение процесса при запуске. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин динамическая проверка я использую не как непереведенное слово, а как название конкретного объекта. По-русски это динамическая проверка; в проекте это проверяется

через отчет, JSON и ручной переход к адресу или событию'.

16. признак: признак

Переводить признак надо не одним словом, а смыслом. В этой книге рабочий русский вариант - признак. Коротко: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин признак я использую не как непереведенное слово, а как название конкретного объекта. По-русски это признак; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

17. уверенность: уверенность

Переводить уверенность надо не одним словом, а смыслом. В этой книге рабочий русский вариант - уверенность. Коротко: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин уверенность я использую не как непереведенное слово, а как название конкретного объекта. По-русски это уверенность; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

18. Ghidra: среда реверс-инжиниринга

Переводить Ghidra надо не одним словом, а смыслом. В этой книге рабочий русский вариант - среда реверс-инжиниринга. Коротко: инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин Ghidra я использую не как непереведенное слово, а как название конкретного объекта. По-русски это среда реверс-инжиниринга; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

19. отладчик: отладчик

Переводить отладчик надо не одним словом, а смыслом. В этой книге рабочий русский вариант - отладчик. Коротко: инструмент управления процессом и получения события отладки. В динамике отладчик позволяет увидеть exception события, точки останова и срабатывания обработчиков.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин отладчик я использую не как непереведенное слово, а как название конкретного объекта. По-русски это отладчик; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

20. журнал выполнения: журнал выполнения

Переводить журнал выполнения надо не одним словом, а смыслом. В этой книге рабочий русский вариант - журнал выполнения. Коротко: последовательность событий, собранных во время запуска. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин журнал выполнения я использую не как непереведенное слово, а как название конкретного объекта. По-русски это журнал выполнения; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

21. JSON: структурированный отчет

Переводить JSON надо не одним словом, а смыслом. В этой книге рабочий русский вариант - структурированный отчет. Коротко: машиночитаемый формат полей анализа. JSON помогает проверить точные поля без влияния HTML-верстки.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин JSON я использую не как непереведенное слово, а как название конкретного объекта. По-русски это структурированный отчет; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

22. HTML: читаемый отчет

Переводить HTML надо не одним словом, а смыслом. В этой книге рабочий русский вариант - читаемый отчет. Коротко: страница для просмотра результатов человеком. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON.

Когда этот термин появляется в отчете, не оставляй его английским ярлыком. Сразу добавь русское объяснение: какой объект назван, какой слой анализа затронут, какая проверка нужна. Именно так английская документация превращается в понятный ответ на русском языке.

Фраза для защиты: 'Термин HTML я использую не как непереведенное слово, а как название конкретного объекта. По-русски это читаемый отчет; в проекте это проверяется через отчет, JSON и ручной переход к адресу или событию'.

Главное правило этой главы: английский термин допустим, если сразу раскрыт по-русски и связан с проверкой. Если слово нельзя привязать к адресу, полю отчета, секции, событию или ручному действию, значит оно пока не объяснено.

Том II-B. Сквозной рассказ: как вся система складывается в одну картину

Эта глава написана как единая лекция без таблиц и списков. Ее цель - собрать разрозненные термины в последовательную историю. Если предыдущие главы объясняли детали, здесь мы идем по всей цепочке целиком: файл, память, функция, стек, исключение, метаданные, отчет, Ghidra и защита результата.

1. От файла на диске к образу в памяти

Файл на диске кажется неподвижным, но Windows читает его как инструкцию к построению образа. PE-заголовки объясняют, какие диапазоны байтов станут секциями, какие права получат эти секции и где искать служебные каталоги. Это первый большой переход: байты перестают быть просто байтами и становятся частями образа.

Если этот переход не понять, дальше начинается путаница. Человек видит число и не знает, является ли оно смещением в файле, RVA или адресом в памяти. Поэтому любой анализ EH/Unwind начинается с дисциплины адресов. Нельзя декодировать метаданные, пока не понятно, где она физически лежит и как она попадает в image.

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию или ручной проверке.

2. От образа к функции

Функция в бинарнике не имеет красивой рамки, как в исходном коде. Ее границы восстанавливаются через дизассемблерное представление, symbol information, метаданные или эвристики. На Windows x64 важную роль играет RUNTIME_FUNCTION: она задает диапазон кода и ссылку на UNWIND_INFO.

Именно здесь проект становится специализированным. Он не пытается понять всю бизнес-логику программы. Он спрашивает: как Windows будет понимать функции, если произойдет исключение? Где начинается диапазон, где заканчивается, и где лежит инструкция по восстановлению стека?

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию или ручной проверке.

3. От функции к стеку

Каждая функция оставляет след в стеке. Она может выделить место под локальные данные, сохранить регистры, вызвать другие функции и вернуть управление. ABI задает правила, чтобы разные функции могли взаимодействовать без разрушения состояния.

Unwind метаданные существует потому, что при исключении система должна пройти этот путь назад. Если функция изменила RSP, Windows должна знать, как вернуть прежнее состояние. Поэтому UNWIND_INFO является не абстрактной таблицей, а кратким описанием того, как функция обращается со стеком.

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию

или ручной проверке.

4. От стека к исключению

Исключение останавливает обычный ход выполнения. У системы появляется вопрос: кто должен обработать событие и как безопасно восстановить состояние? Для ответа нужны context, RIP, stack, function table и обработчик. Это не одна структура, а цепочка решений.

Если в этой цепочке поврежден адрес, странно размещена метаданные или обработчик указывает в подозрительную область, анализатор фиксирует не эмоцию, а нарушение условий, при которых цепочка должна быть понятной.

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию или ручной проверке.

5. От статического факта к динамическому событию

Статический анализ видит карту, записанную в файле. Динамический анализ видит отдельное путешествие по этой карте. Карта может содержать необычность, даже если конкретный запуск до нее не дошел. И наоборот, запуск может показать регистрацию dynamic function table, которую статическая карта не описывает полностью.

Поэтому сильный вывод строится на сопоставлении: что лежит в файле, что произошло при запуске, где эти два слоя подтверждают друг друга, а где остаются ограничениями эксперимента.

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию или ручной проверке.

6. От признак к человеческому выводу

Признак является названием наблюдения. Он нужен, чтобы не писать каждый раз длинное описание нарушения. Но признак не должен становиться ярлыком без смысла. За ним всегда должны стоять объект, адрес, правило и проверка.

Хороший отчет помогает человеку пройти этот путь: Final assessment дает обзор, Static признаки дают список фактов, Функции времени выполнения дают строки таблицы, Журнал выполнения показывает события, Manual validation plan говорит, что проверить руками.

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию или ручной проверке.

7. От Ghidra к доказательству

Ghidra превращает байты в карту кода и данных, но она не является последней инстанцией. Ее сила в том, что она помогает человеку проверить конкретный вопрос: что находится по этому адресу, как выглядит surrounding дизассемблерное представление, где секция и какие есть ссылки.

Если отчет говорит о representative RVA, Ghidra дает способ показать это место. Если отчет говорит о обработчик, Ghidra помогает увидеть код. Если Ghidra не согласна с границами функции, это не конец анализа, а начало сравнения двух моделей.

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию или ручной проверке.

8. От документации к своей речи

Документация Microsoft точна, но написана не как учебник для нулевого читателя. Она задает поля, структуры и API. Задача студента - превратить эту точность в понятную речь: сначала русское объяснение, потом английский термин, потом техническое поле, потом пример из проекта.

Именно так литература и документация становятся частью диплома. Не надо пересказывать Microsoft Learn абзацами. Надо показать, что правила из документации объясняют, почему анализатор проверяет именно эти поля и почему признак имеет технический смысл.

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию или ручной проверке.

9. От отчета к презентации

Презентация не должна показывать все 500 страниц. Она должна показать маршрут мышления. Один benign образец нужен как контроль. Один комбинированный образец нужен как главный рассказ. Один переход в Ghidra нужен как ручное подтверждение. Одна команда запуска нужна как воспроизводимость.

Если все это есть, проект выглядит зрелым. Он не обещает невозможного, не делает вид, что один запуск знает все поведение, и не прячет детали. Он показывает инструмент, метод и границы вывода.

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию

или ручной проверке.

10. От новичка к реверс-инженеру

Рост в реверс-инжиниринге происходит не тогда, когда человек запоминает больше страшных слов. Рост происходит тогда, когда он учится задавать точные вопросы. Что я вижу? Где это лежит? Кто это создал? Кто это использует? Как проверить? Чего это не доказывает?

EH/Unwind Analyzer является хорошей учебной темой именно потому, что заставляет удерживать много слоев сразу: файл, PE, память, ABI, стек, исключения, метаданные, статический отчет, динамический запуск и Ghidra. Когда эти слои складываются, появляется настоящее понимание.

Проверь понимание этого шага на любом отчете проекта: найди одно поле, которое относится к теме, и произнеси его смысл по-русски. Если в объяснении остался только английский термин, значит шаг надо перечитать и привязать к адресу, секции, событию или ручной проверке.

Итог этой главы простой: теория нужна не для украшения диплома, а для того, чтобы не делать ложных скачков. Сильный реверс-инженер не угадывает смысл бинарника. Он восстанавливает цепочку договоров, проверяет каждый переход и честно говорит, где вывод силен, а где требует дополнительного эксперимента.

Том II. Живой словарь: термины не отдельно, а во взаимосвязи

Технический словарь часто пугает новичка тем, что каждое слово кажется отдельной дверью в новую комнату. В реверс-инжиниринге это ощущение особенно сильное: byte, RVA, VA, секция, обработчик, unwind метаданные, exception directory, уверенность. Но на самом деле эти слова образуют цепочку. Одно понятие подает руку следующему.

Поэтому ниже термины объясняются не как сухой глоссарий. Каждый термин получает простой смысл, строгий смысл, место в проекте, соседние понятия и ручную проверку. Это не справочник для заучивания. Это тренажер связного мышления.

1. байт (byte)

Байт сначала надо понять по-человечески: минимальный кусок файла, который можно адресовать и показать в hex viewer. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это 8 бит данных; именно из байтов состоят PE headers, секции, машинные инструкции и unwind метаданные. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: Buf в проекте хранит исследуемый EXE как массив байтов. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: файловое смещение, hex, парсер. Эти связи важнее голого перевода. Например, если термин связан с адресами,

нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: открыть EXE в hex viewer и найти первые байты MZ. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое byte', а показывает, где это находится и как проверить.

2. шестнадцатеричная запись (hex)

Шестнадцатеричная запись сначала надо понять по-человечески: способ писать байты и адреса компактно. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это система счисления с основанием 16, где одна цифра кодирует 4 бита. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: отчеты печатают RVA, VA, offsets и sizes как 0x1000, 0x7100, 0x140000000. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: byte, address, RVA. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: перевести 0x10, 0x100 и 0x1000 в десятичные числа. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое hex', а показывает, где это находится и как проверить.

3. файловое смещение (файловое смещение)

Файловое смещение сначала надо понять по-человечески: координата байта от начала файла на диске. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это целое число, показывающее позицию в raw file representation. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: нужно, чтобы безопасно читать структуру после перевода RVA в offset. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: RVA, секция, raw size. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: вычислить offset = PointerToRawData + (RVA - VirtualAddress). Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое файловое смещение', а показывает, где это находится и как проверить.

4. RVA (Relative Virtual Address)

Rva сначала надо понять по-человечески: адрес внутри образа без учета того, куда Windows загрузила файл. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это image-relative coordinate; RVA прибавляется к loaded ImageBase для получения VA. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: RUNTIME_FUNCTION и UNWIND_INFO используют RVA. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: VA, ImageBase, секция. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: найти RVA в отчете и перейти в Ghidra как ImageBase + RVA. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое Relative Virtual Address', а показывает, где это находится и как проверить.

5. VA (Virtual Address)

Va сначала надо понять по-человечески: адрес в памяти конкретного запущенного процесса. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это координата внутри virtual address space после загрузки image. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: dynamic log показывает VA обработчиков, точки останова и API. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: RVA, ASLR, процесс. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: сравнить loaded base в журнал выполнения и адрес обработчик. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое Virtual Address', а показывает, где это находится и как проверить.

6. секция PE (секция)

Секция pe сначала надо понять по-человечески: именованный диапазон файла и памяти с ролью и правами. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это секция header задает virtual range, raw backing, name и characteristics. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: анализатор проверяет .text, .pdata, .xdata и необычные секции. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: permissions, raw size, virtual size. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: открыть таблицу Sections в HTML-отчете. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое секция', а показывает, где это находится и как проверить.

7. права секции (permissions)

Права секции сначала надо понять по-человечески: ответ на вопрос: можно ли область читать, писать или исполнять. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это битовые признаки секции R, W, X в PE characteristics. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: writable или executable placement для unwind метаданные влияет на признаки. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: секция, метаданные placement, обработчик. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: найти RW или R-X секцию и объяснить риск. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое permissions', а показывает, где это находится и как проверить.

8. PE (Portable Executable)

Pe сначала надо понять по-человечески: формат Windows EXE и DLL. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это контракт между компоновщиком, загрузчиком Windows и инструментами анализа. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: pe.c строит модель PE без запуска целевая программа. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: headers, sections, каталоги данных. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: найти DOS header, PE signature и таблица секций. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое Portable Executable', а показывает, где это находится и как проверить.

9. каталог данных (data directory)

Каталог данных сначала надо понять по-человечески: таблица ссылок на важные области PE. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это массив RVA/size пар в OptionalHeader. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: exception directory index 3 ведет к RUNTIME_FUNCTION table. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: OptionalHeader, exception directory, RVA. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: найти Exception Directory и проверить его размер. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое data directory', а показывает, где это находится и как проверить.

10. каталог исключений (exception directory)

Каталог исключений сначала надо понять по-человечески: указатель на таблицу функций для обработки исключений x64. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это data directory, который для AMD64 описывает массив RUNTIME_FUNCTION. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: если каталог выходит за image, появляется pe_exception_directory_oob. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: .pdata, RUNTIME_FUNCTION, SizeOfImage. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: сравнить directory RVA/size с секциями. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое exception directory', а показывает, где это находится и как проверить.

11. RUNTIME_FUNCTION (время выполнения function entry)

Runtime_function сначала надо понять по-человечески: строка таблицы: где начинается функция, где кончается и где ее unwind info. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это три 32-битных image-relative поля: BeginAddress, EndAddress, UnwindInfoAddress. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: главный объект таблицы Функции времени выполнения в отчете. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: UNWIND_INFO, .pdata, function range. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: выбрать строку Функции времени выполнения и объяснить begin/end/unwind RVA. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое время выполнения function entry', а показывает, где это находится и как проверить.

12. UNWIND_INFO (unwind метаданные)

Unwind_info сначала надо понять по-человечески: рецепт, как восстановить стек функции при исключении. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это структура Windows x64 с version, flags, prolog size, unwind codes и optional обработчик/chained info. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: unwind.c декодирует структуру и проверяет границы. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: prolog, кадр стека, обработчик. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: сравнить Version/flags и Обработчик в отчете. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое unwind метаданные', а показывает, где это находится и как проверить.

13. стек (stack)

Стек сначала надо понять по-человечески: область памяти для истории вызовов функций. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это LIFO-память потока с return address, saved registers и local variables. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: unwind метаданные нужна, чтобы идти от текущего кадра к предыдущему. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: call, ret, кадр стека. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: нарисовать stack до call, после call и после prolog. Это превращает термин из слова в действие. Реверс-инженер не

просто говорит 'я знаю, что такое stack', а показывает, где это находится и как проверить.

14. кадр стека (кадр стека)

Кадр стека сначала надо понять по-человечески: кусок стека, принадлежащий одной функции. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это логическая область с адресом возврата, сохраненными регистрами и локальными данными. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: UNWIND_INFO описывает, как восстановить прошлый frame. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: RSP, prolog, epilog. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: найти sub rsp в дизассемблерное представление и объяснить выделение frame. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое кадр стека', а показывает, где это находится и как проверить.

15. ABI (Application Binary Interface)

Abi сначала надо понять по-человечески: правила общения функций на машинном уровне. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это соглашение о регистрах аргументов, сохранении регистров и выравнивании стека. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: Windows x64 ABI объясняет смысл unwind codes. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: registers, prolog, stack alignment. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: объяснить shadow space и 16-byte alignment. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое Application Binary Interface', а показывает, где это находится и как проверить.

16. исключение (exception)

Исключение сначала надо понять по-человечески: событие, которое ломает обычный ход выполнения. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это механизм Windows для ошибок, точки останова и dispatch к обработчикам. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: dynamic mode собирает exception события. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: SEH, VEH, обработчик. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: найти first chance и second chance в журнал выполнения. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое exception', а показывает, где это находится и как проверить.

17. обработчик (обработчик)

Обработчик сначала надо понять по-человечески: код, которому дают шанс обработать исключение. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это function или обратный вызов, связанный с dispatch исключения. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: обработчик целевая программа может быть в UNWIND_INFO, VEH или UEF. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: VEH, UEF, обработчик dominance. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: перейти в Ghidra по обработчик address. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое обработчик', а показывает, где это находится и как проверить.

18. статический анализ (статический анализ)

Статический анализ сначала надо понять по-человечески: чтение файла без запуска. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это исследование bytes, headers, tables и метаданные как данных. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: основной режим анализатор строит признаки. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: парсер, признаки, report. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно

сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: запустить анализатор без --dynamic. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое статический анализ', а показывает, где это находится и как проверить.

19. динамическая проверка (динамическая проверка)

Динамическая проверка сначала надо понять по-человечески: наблюдение за целевая программа при запуске. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это Debug API создает процесс, ловит события, ставит точки останова и собирает свидетельства времени выполнения. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: dynamic_win.c фиксирует registrations, exceptions и срабатывания обработчиков. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: debug event, свидетельства времени выполнения, тайм-аут. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: запустить --dynamic и сравнить Журнал выполнения с static признаки. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое динамическая проверка', а показывает, где это находится и как проверить.

20. признак (признак)

Признак сначала надо понять по-человечески: структурированное сообщение о низкоуровневом факте. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это code, family, severity, affected count, representative RVA и message. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: heuristics.c превращает наблюдения парсер в признаки. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: оценка, уверенность, свидетельства. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: найти признак в JSON и объяснить его без слова 'вредоносно'. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое признак', а показывает, где это находится и как проверить.

21. уверенность (уверенность интерпретации)

Уверенность сначала надо понять по-человечески: насколько устойчиво объяснение найденных фактов. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это инженерная оценка, а не статистическая вероятность. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: assessment использует уверенность вместе со оценка и свидетельства level. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: оценка, mitigation, case strength. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: объяснить, почему уверенность не равен вероятности malware. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое уверенность интерпретации', а показывает, где это находится и как проверить.

22. Ghidra (Ghidra)

Ghidra сначала надо понять по-человечески: инструмент, где байты становятся кодом, функциями и комментариями. В этот момент не нужно стесняться простого объяснения. Хороший преподаватель всегда начинает с него, потому что без простого образа строгая формула остается чужой.

Строго говоря, это реверс-инжиниринг suite с дизассемблерное представление, decompiler, labels и bookmarks. Здесь важно не только определение, но и границы применимости. Почти все ошибки новичка связаны с тем, что он переносит смысл термина в неправильный слой: файл, image, процесс, отчет или Ghidra.

В EH/Unwind Analyzer этот термин появляется так: Ghidra CSV помогает перейти от report к адресам. Значит, когда ты видишь его в коде или отчете, надо сразу спросить: из какого слоя пришло это значение, какая функция проекта его создала и какая функция потом ему доверяет.

Соседние понятия: дизассемблерное представление, function, manual validation. Эти связи важнее голого перевода. Например, если термин связан с адресами, нужно сразу отличать файловое смещение, RVA и VA. Если связан с обработка исключений, нужно вспомнить стек, frame и обработчик.

Ручная проверка: импортировать целевая программа и перейти к ImageBase + RVA. Это превращает термин из слова в действие. Реверс-инженер не просто говорит 'я знаю, что такое Ghidra', а показывает, где это находится и как проверить.

Том III. Лекция 1. С нуля: зачем реверсеру вообще думать слоями

Эта лекция нужна, чтобы разобрать тему 'переход от обычного файла к системе договоров' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `byte` в этой книге читается так: по-русски - байт. Короткий смысл: минимальная единица данных в файле. Байт сам по себе не имеет смысла, пока мы не знаем, является ли он частью инструкции, числа, строки, поля PE или служебного выравнивания. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `byte` глубже. Русская опора - байт. Если убрать технический шум, то перед нами минимальная единица данных в файле. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'С нуля: зачем реверсеру вообще думать слоями' этот термин связан с общей цепочкой анализа. Байт сам по себе не имеет смысла, пока мы не знаем, является ли он частью инструкции, числа, строки, поля PE или служебного выравнивания. Если термин

встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `byte` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин файловое смещение в этой книге читается так: по-русски - файловое смещение. Короткий смысл: позиция байта от начала файла на диске. Это координата hex viewer. Она нужна, когда мы доказываем, что структура действительно читается из файла. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем файловое смещение глубже. Русская опора - файловое смещение. Если убрать технический шум, то перед нами позиция байта от начала файла на диске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'С нуля: зачем реверсеру вообще думать слоями' этот термин связан с общей цепочкой анализа. Это координата hex viewer. Она нужна, когда мы доказываем, что структура действительно читается из файла. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю,

какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину файловое смещение простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин PE в этой книге читается так: по-русски - Portable Executable. Короткий смысл: формат исполняемых файлов Windows. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем PE глубже. Русская опора - Portable Executable. Если убрать технический шум, то перед нами формат исполняемых файлов Windows. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'С нуля: зачем реверсеру вообще думать слоями' этот термин связан с общей цепочкой анализа. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `PE` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин секция в этой книге читается так: по-русски - секция PE. Короткий смысл: именованный диапазон файла и памяти с правами доступа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем секция глубже. Русская опора - секция PE. Если убрать технический шум, то перед нами именованный диапазон файла и памяти с правами доступа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'С нуля: зачем реверсеру вообще думать слоями' этот термин связан с общей цепочкой анализа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину секция простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'С нуля: зачем реверсеру вообще думать слоями' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'С нуля: зачем реверсеру вообще думать слоями' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как

последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал,

сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю переход от обычного файла к системе договоров. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть

шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 2. От исходного текста к работающему процессу

Эта лекция нужна, чтобы разобрать тему 'как идея программиста превращается в машинный код и память процесса' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `ABI` в этой книге читается так: по-русски - двоичный интерфейс приложения. Короткий смысл: правила вызова функций, регистров, стека и выравнивания. Без ABI нельзя понять prolog, epilog, кадр стека и смысл UNWIND_INFO. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `ABI` глубже. Русская опора - двоичный интерфейс приложения. Если убрать технический шум, то перед нами правила вызова функций, регистров, стека и выравнивания. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'От исходного текста к работающему процессу' этот термин связан с общей цепочкой анализа. Без ABI нельзя понять prolog, epilog, кадр стека и смысл UNWIND_INFO. Если термин

встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `ABI` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `stack` в этой книге читается так: по-русски - стек. Короткий смысл: память потока для вызовов функций и временного состояния. Стек связывает call/return, локальные данные, сохраненные регистры и обработку исключений. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `stack` глубже. Русская опора - стек. Если убрать технический шум, то перед нами память потока для вызовов функций и временного состояния. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'От исходного текста к работающему процессу' этот термин связан с общей цепочкой анализа. Стек связывает call/return, локальные данные, сохраненные регистры и обработку исключений. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это

поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `stack` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин VA в этой книге читается так: по-русски - виртуальный адрес. Короткий смысл: адрес в памяти конкретного процесса. VA получается из ImageBase и RVA; именно такие адреса часто видны в отладчик и Ghidra. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем VA глубже. Русская опора - виртуальный адрес. Если убрать технический шум, то перед нами адрес в памяти конкретного процесса. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'От исходного текста к работающему процессу' этот термин связан с общей цепочкой анализа. VA получается из ImageBase и RVA; именно такие адреса часто видны в отладчик и Ghidra. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину VA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин отладчик в этой книге читается так: по-русски - отладчик. Короткий смысл: инструмент управления процессом и получения события отладки. В динамике отладчик позволяет увидеть exception события, точки останова и срабатывания обработчиков. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем отладчик глубже. Русская опора - отладчик. Если убрать технический шум, то перед нами инструмент управления процессом и получения события отладки. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'От исходного текста к работающему процессу' этот термин связан с общей цепочкой анализа. В динамике отладчик позволяет увидеть exception события, точки останова и срабатывания обработчиков. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину отладчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин журнал выполнения в этой книге читается так: по-русски - журнал выполнения. Короткий смысл: последовательность событий, собранных во время запуска. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем журнал выполнения глубже. Русская опора - журнал выполнения. Если убрать технический шум, то перед нами последовательность событий, собранных во время запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'От исходного текста к работающему процессу' этот термин связан с общей цепочкой анализа. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину журнал выполнения простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'От исходного текста к работающему процессу' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как

последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал,

сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как идея программиста превращается в машинный код и память процесса. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда

слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 3. Данные, память и адреса без путаницы

Эта лекция нужна, чтобы разобрать тему 'почему файловое смещение, RVA и VA нельзя смешивать' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин файловое смещение в этой книге читается так: по-русски - файловое смещение. Короткий смысл: позиция байта от начала файла на диске. Это координата hex viewer. Она нужна, когда мы доказываем, что структура действительно читается из файла. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем файловое смещение глубже. Русская опора - файловое смещение. Если убрать технический шум, то перед нами позиция байта от начала файла на диске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Данные, память и адреса без путаницы' этот термин связан с общей цепочкой анализа. Это координата hex viewer. Она нужна, когда мы доказываем, что структура действительно читается из файла. Если термин встречается в отчете, его надо сразу

перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину файловое смещение простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин RVA в этой книге читается так: по-русски - относительный виртуальный адрес. Короткий смысл: адрес внутри загруженного образа относительно ImageBase. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем RVA глубже. Русская опора - относительный виртуальный адрес. Если убрать технический шум, то перед нами адрес внутри загруженного образа относительно ImageBase. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Данные, память и адреса без путаницы' этот термин связан с общей цепочкой анализа. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю,

какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину RVA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин VA в этой книге читается так: по-русски - виртуальный адрес. Короткий смысл: адрес в памяти конкретного процесса. VA получается из ImageBase и RVA; именно такие адреса часто видны в отладчик и Ghidra. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем VA глубже. Русская опора - виртуальный адрес. Если убрать технический шум, то перед нами адрес в памяти конкретного процесса. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Данные, память и адреса без путаницы' этот термин связан с общей цепочкой анализа. VA получается из ImageBase и RVA; именно такие адреса часто видны в отладчик и Ghidra. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину VA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин секция в этой книге читается так: по-русски - секция PE. Короткий смысл: именованный диапазон файла и памяти с правами доступа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем секция глубже. Русская опора - секция PE. Если убрать технический шум, то перед нами именованный диапазон файла и памяти с правами доступа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Данные, память и адреса без путаницы' этот термин связан с общей цепочкой анализа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `секция` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `Ghidra` в этой книге читается так: по-русски - среда реверс-инжиниринга. Короткий смысл: инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `Ghidra` глубже. Русская опора - среда реверс-инжиниринга. Если убрать технический шум, то перед нами инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Данные, память и адреса без путаницы' этот термин связан с общей цепочкой анализа. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `Ghidra` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Данные, память и адреса без путаницы' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как

последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал,

сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю почему файловое смещение, RVA и VA нельзя смешивать. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда

слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 4. Процессор, функция, стек и соглашение вызовов

Эта лекция нужна, чтобы разобрать тему 'как CPU, ABI и кадр стека делают функцию понятной' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `ABI` в этой книге читается так: по-русски - двоичный интерфейс приложения. Короткий смысл: правила вызова функций, регистров, стека и выравнивания. Без ABI нельзя понять prolog, epilog, кадр стека и смысл UNWIND_INFO. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `ABI` глубже. Русская опора - двоичный интерфейс приложения. Если убрать технический шум, то перед нами правила вызова функций, регистров, стека и выравнивания. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Процессор, функция, стек и соглашение вызовов' этот термин связан с общей цепочкой анализа. Без ABI нельзя понять prolog, epilog, кадр стека и смысл UNWIND_INFO. Если термин встречается в отчете, его надо сразу перевести в русский инженерный

вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `ABI` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `stack` в этой книге читается так: по-русски - стек. Короткий смысл: память потока для вызовов функций и временного состояния. Стек связывает call/return, локальные данные, сохраненные регистры и обработку исключений. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `stack` глубже. Русская опора - стек. Если убрать технический шум, то перед нами память потока для вызовов функций и временного состояния. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Процессор, функция, стек и соглашение вызовов' этот термин связан с общей цепочкой анализа. Стек связывает call/return, локальные данные, сохраненные регистры и обработку исключений. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `stack` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `frame` в этой книге читается так: по-русски - кадр стека. Короткий смысл: участок стека, принадлежащий конкретной функции. Unwind метаданные нужна, чтобы восстановить предыдущий frame при исключении. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `frame` глубже. Русская опора - кадр стека. Если убрать технический шум, то перед нами участок стека, принадлежащий конкретной функции. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Процессор, функция, стек и соглашение вызовов' этот термин связан с общей цепочкой анализа. Unwind метаданные нужна, чтобы восстановить предыдущий frame при исключении. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для

обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `frame` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть

динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `UNWIND_INFO` в этой книге читается так: по-русски - информация раскрутки стека. Короткий смысл: структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `UNWIND_INFO` глубже. Русская опора - информация раскрутки стека. Если убрать технический шум, то перед нами структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Процессор, функция, стек и соглашение вызовов' этот термин связан с общей цепочкой анализа. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время

запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `UNWIND_INFO` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала

статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин обработчик в этой книге читается так: по-русски - обработчик исключения. Короткий смысл: код или обратный вызов, которому может быть передано управление. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем обработчик глубже. Русская опора - обработчик исключения. Если убрать технический шум, то перед нами код или обратный вызов, которому может быть передано управление. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Процессор, функция, стек и соглашение вызовов' этот термин связан с общей цепочкой анализа. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для

обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину обработчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть

динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Процессор, функция, стек и соглашение вызовов' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова

функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если

есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как CPU, ABI и кадр стека делают функцию понятной. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 5. Windows: процесс, поток, исключение

Эта лекция нужна, чтобы разобрать тему 'как операционная система видит выполнение и особые события' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин отладчик в этой книге читается так: по-русски - отладчик. Короткий смысл: инструмент управления процессом и получения события отладки. В динамике отладчик позволяет увидеть exception события, точки останова и срабатывания обработчиков. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем отладчик глубже. Русская опора - отладчик. Если убрать технический шум, то перед нами инструмент управления процессом и получения события отладки. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Windows: процесс, поток, исключение' этот термин связан с общей цепочкой анализа. В динамике отладчик позволяет увидеть exception события, точки останова и срабатывания

обработчиков. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину отладчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к

динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин журнал выполнения в этой книге читается так: по-русски - журнал выполнения. Короткий смысл: последовательность событий, собранных во время запуска. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем журнал выполнения глубже. Русская опора - журнал выполнения. Если убрать технический шум, то перед нами последовательность событий, собранных во время запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Windows: процесс, поток, исключение' этот термин связан с общей цепочкой анализа. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения.

Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину журнал выполнения простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин обработчик в этой книге читается так: по-русски - обработчик исключения. Короткий смысл: код или обратный вызов, которому может быть передано управление. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем обработчик глубже. Русская опора - обработчик исключения. Если убрать технический шум, то перед нами код или обратный вызов, которому может быть передано управление. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Windows: процесс, поток, исключение' этот термин связан с общей цепочкой анализа. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он

динамически. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину обработчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к

динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин динамическая проверка в этой книге читается так: по-русски - динамическая проверка. Короткий смысл: наблюдение процесса при запуске. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем динамическая проверка глубже. Русская опора - динамическая проверка. Если убрать технический шум, то перед нами наблюдение процесса при запуске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Windows: процесс, поток, исключение' этот термин связан с общей цепочкой анализа. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Если термин встречается в отчете, его надо сразу

перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину динамическая проверка простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Windows: процесс, поток, исключение' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Windows: процесс, поток, исключение' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как

последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал,

сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как операционная система видит выполнение и особые события. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда

слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 6. PE как карта загрузчика Windows

Эта лекция нужна, чтобы разобрать тему 'как секции и каталоги данных связывают файл с памятью' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин PE в этой книге читается так: по-русски - Portable Executable. Короткий смысл: формат исполняемых файлов Windows. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем PE глубже. Русская опора - Portable Executable. Если убрать технический шум, то перед нами формат исполняемых файлов Windows. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE как карта загрузчика Windows' этот термин связан с общей цепочкой анализа. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или

диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `PE` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин секция в этой книге читается так: по-русски - секция PE. Короткий смысл: именованный диапазон файла и памяти с правами доступа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем секция глубже. Русская опора - секция PE. Если убрать технический шум, то перед нами именованный диапазон файла и памяти с правами доступа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE как карта загрузчика Windows' этот термин связан с общей цепочкой анализа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это

поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `секция` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин RVA в этой книге читается так: по-русски - относительный виртуальный адрес. Короткий смысл: адрес внутри загруженного образа относительно ImageBase. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем RVA глубже. Русская опора - относительный виртуальный адрес. Если убрать технический шум, то перед нами адрес внутри загруженного образа относительно ImageBase. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE как карта загрузчика Windows' этот термин связан с общей цепочкой анализа. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину RVA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин метаданные в этой книге читается так: по-русски - метаданные. Короткий смысл: данные, описывающие другие данные или код. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем метаданные глубже. Русская опора - метаданные. Если убрать технический шум, то перед нами данные, описывающие другие данные или код. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE как карта загрузчика Windows' этот термин связан с общей цепочкой анализа. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить

файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину метаданные простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по

адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `RUNTIME_FUNCTION` в этой книге читается так: по-русски - запись таблицы функций времени выполнения. Короткий смысл: тройка BeginAddress, EndAddress и UnwindInfoAddress. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `RUNTIME_FUNCTION` глубже. Русская опора - запись таблицы функций времени выполнения. Если убрать технический шум, то перед нами тройка BeginAddress, EndAddress и UnwindInfoAddress. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE как карта загрузчика Windows' этот термин связан с общей цепочкой анализа. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для

обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `RUNTIME_FUNCTION` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть

динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'PE как карта загрузчика Windows' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова

функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если

есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как секции и каталоги данных связывают файл с памятью. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 7. Каталог исключений и таблица RUNTIME_FUNCTION

Эта лекция нужна, чтобы разобрать тему 'почему Windows нужен каталог функций для обработки исключений x64' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `RUNTIME_FUNCTION` в этой книге читается так: по-русски - запись таблицы функций времени выполнения. Короткий смысл: тройка BeginAddress, EndAddress и UnwindInfoAddress. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `RUNTIME_FUNCTION` глубже. Русская опора - запись таблицы функций времени выполнения. Если убрать технический шум, то перед нами тройка BeginAddress, EndAddress и UnwindInfoAddress. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Каталог исключений и таблица RUNTIME_FUNCTION' этот термин связан с общей цепочкой анализа.

Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `RUNTIME_FUNCTION` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком,

прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `UNWIND_INFO` в этой книге читается так: по-русски - информация раскрутки стека. Короткий смысл: структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `UNWIND_INFO` глубже. Русская опора - информация раскрутки стека. Если убрать технический шум, то перед нами структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Каталог исключений и таблица RUNTIME_FUNCTION' этот термин связан с общей цепочкой анализа.

Она объясняет системе, как безопасно вернуться к вызывающему кадру. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `UNWIND_INFO` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к

динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин RVA в этой книге читается так: по-русски - относительный виртуальный адрес. Короткий смысл: адрес внутри загруженного образа относительно ImageBase. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем RVA глубже. Русская опора - относительный виртуальный адрес. Если убрать технический шум, то перед нами адрес внутри загруженного образа относительно ImageBase. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Каталог исключений и таблица RUNTIME_FUNCTION' этот термин связан с общей цепочкой анализа. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Если термин встречается в отчете, его надо сразу

перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину RVA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин секция в этой книге читается так: по-русски - секция PE. Короткий смысл: именованный диапазон файла и памяти с правами доступа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем секция глубже. Русская опора - секция PE. Если убрать технический шум, то перед нами именованный диапазон файла и памяти с правами доступа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Каталог исключений и таблица RUNTIME_FUNCTION' этот термин связан с общей цепочкой анализа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое

поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `секция` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Каталог исключений и таблица RUNTIME_FUNCTION' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что

изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Каталог исключений и таблица RUNTIME_FUNCTION' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю почему Windows нужен каталог функций для обработки исключений x64. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 8. UNWIND_INFO как язык восстановления стека

Эта лекция нужна, чтобы разобрать тему 'почему prolog, flags и обработчик превращаются в структуру' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `UNWIND_INFO` в этой книге читается так: по-русски - информация раскрутки стека. Короткий смысл: структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `UNWIND_INFO` глубже. Русская опора - информация раскрутки стека. Если убрать технический шум, то перед нами структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'UNWIND_INFO как язык восстановления стека' этот термин связан с общей цепочкой анализа. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Если термин

встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `UNWIND_INFO` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `ABI` в этой книге читается так: по-русски - двоичный интерфейс приложения. Короткий смысл: правила вызова функций, регистров, стека и выравнивания. Без ABI нельзя понять prolog, epilog, кадр стека и смысл UNWIND_INFO. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `ABI` глубже. Русская опора - двоичный интерфейс приложения. Если убрать технический шум, то перед нами правила вызова функций, регистров, стека и выравнивания. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'UNWIND_INFO как язык восстановления стека' этот термин связан с общей цепочкой анализа. Без ABI нельзя понять prolog, epilog, кадр стека и смысл UNWIND_INFO. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним

стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `ABI` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `stack` в этой книге читается так: по-русски - стек. Короткий смысл: память потока для вызовов функций и временного состояния. Стек связывает call/return, локальные данные, сохраненные регистры и обработку исключений. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `stack` глубже. Русская опора - стек. Если убрать технический шум, то перед нами память потока для вызовов функций и временного состояния. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'UNWIND_INFO как язык восстановления стека' этот термин связан с общей цепочкой анализа. Стек связывает call/return, локальные данные, сохраненные регистры и обработку исключений. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `stack` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `frame` в этой книге читается так: по-русски - кадр стека. Короткий смысл: участок стека, принадлежащий конкретной функции. Unwind метаданные нужна, чтобы восстановить предыдущий frame при исключении. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `frame` глубже. Русская опора - кадр стека. Если убрать технический шум, то перед нами участок стека, принадлежащий конкретной функции. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'UNWIND_INFO как язык восстановления стека' этот термин связан с общей цепочкой анализа. Unwind метаданные нужна, чтобы восстановить предыдущий frame при исключении. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для

обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `frame` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть

динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин обработчик в этой книге читается так: по-русски - обработчик исключения. Короткий смысл: код или обратный вызов, которому может быть передано управление. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем обработчик глубже. Русская опора - обработчик исключения. Если убрать технический шум, то перед нами код или обратный вызов, которому может быть передано управление. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'UNWIND_INFO как язык восстановления стека' этот термин связан с общей цепочкой анализа. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить

файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину обработчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по

адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'UNWIND_INFO как язык восстановления стека' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в

памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть

секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю почему prolog, flags и обработчик превращаются в структуру. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 9. Метод реверс-инжиниринга: от наблюдения к выводу

Эта лекция нужна, чтобы разобрать тему 'как не прыгать от странности к обвинению' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `статический анализ` в этой книге читается так: по-русски - статический анализ. Короткий смысл: чтение файла без запуска. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `статический анализ` глубже. Русская опора - статический анализ. Если убрать технический шум, то перед нами чтение файла без запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Метод реверс-инжиниринга: от наблюдения к выводу' этот термин связан с общей цепочкой анализа. Он силен структурной полнотой, но не доказывает, какой путь выполнения

реально произойдет. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `статический анализ` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к

динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин динамическая проверка в этой книге читается так: по-русски - динамическая проверка. Короткий смысл: наблюдение процесса при запуске. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем динамическая проверка глубже. Русская опора - динамическая проверка. Если убрать технический шум, то перед нами наблюдение процесса при запуске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Метод реверс-инжиниринга: от наблюдения к выводу' этот термин связан с общей цепочкой анализа. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Если термин встречается в отчете, его

надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину динамическая проверка простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Метод реверс-инжиниринга: от наблюдения к выводу' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое

поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину признак простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Метод реверс-инжиниринга: от наблюдения к выводу' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `Ghidra` в этой книге читается так: по-русски - среда реверс-инжиниринга. Короткий смысл: инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `Ghidra` глубже. Русская опора - среда реверс-инжиниринга. Если убрать технический шум, то перед нами инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Метод реверс-инжиниринга: от наблюдения к выводу' этот термин связан с общей цепочкой анализа. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `Ghidra` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Метод реверс-инжиниринга: от наблюдения к выводу' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как

последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал,

сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как не прыгать от странности к обвинению. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть

шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 10. Архитектура EH/Unwind Analyzer

Эта лекция нужна, чтобы разобрать тему 'как проект превращает файл в проверяемый отчет' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин PE в этой книге читается так: по-русски - Portable Executable. Короткий смысл: формат исполняемых файлов Windows. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем PE глубже. Русская опора - Portable Executable. Если убрать технический шум, то перед нами формат исполняемых файлов Windows. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Архитектура EH/Unwind Analyzer' этот термин связан с общей цепочкой анализа. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или

диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `PE` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин метаданные в этой книге читается так: по-русски - метаданные. Короткий смысл: данные, описывающие другие данные или код. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем метаданные глубже. Русская опора - метаданные. Если убрать технический шум, то перед нами данные, описывающие другие данные или код. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Архитектура EH/Unwind Analyzer' этот термин связан с общей цепочкой анализа. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину метаданные простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Архитектура EH/Unwind Analyzer' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить

файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по

адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `JSON` в этой книге читается так: по-русски - структурированный отчет. Короткий смысл: машиночитаемый формат полей анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `JSON` глубже. Русская опора - структурированный отчет. Если убрать технический шум, то перед нами машиночитаемый формат полей анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Архитектура EH/Unwind Analyzer' этот термин связан с общей цепочкой анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `JSON` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин HTML в этой книге читается так: по-русски - читаемый отчет. Короткий смысл: страница для просмотра результатов человеком. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем HTML глубже. Русская опора - читаемый отчет. Если убрать технический шум, то перед нами страница для просмотра результатов человеком. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Архитектура EH/Unwind Analyzer' этот термин связан с общей цепочкой анализа. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает

признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину HTML простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая

граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Архитектура EH/Unwind Analyzer' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как проект превращает файл в проверяемый отчет. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 11. Командная строка и воспроизводимость

Эта лекция нужна, чтобы разобрать тему 'почему команда запуска является частью доказательства' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `статический анализ` в этой книге читается так: по-русски - статический анализ. Короткий смысл: чтение файла без запуска. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `статический анализ` глубже. Русская опора - статический анализ. Если убрать технический шум, то перед нами чтение файла без запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Командная строка и воспроизводимость' этот термин связан с общей цепочкой анализа. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет. Если термин встречается в отчете, его надо сразу

перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `статический анализ` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин динамическая проверка в этой книге читается так: по-русски - динамическая проверка. Короткий смысл: наблюдение процесса при запуске. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем динамическая проверка глубже. Русская опора - динамическая проверка. Если убрать технический шум, то перед нами наблюдение процесса при запуске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Командная строка и воспроизводимость' этот термин связан с общей цепочкой анализа. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю,

какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину динамическая проверка простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `JSON` в этой книге читается так: по-русски - структурированный отчет. Короткий смысл: машиночитаемый формат полей анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `JSON` глубже. Русская опора - структурированный отчет. Если убрать технический шум, то перед нами машиночитаемый формат полей анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Командная строка и воспроизводимость' этот термин связан с общей цепочкой анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `JSON` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин HTML в этой книге читается так: по-русски - читаемый отчет. Короткий смысл: страница для просмотра результатов человеком. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем HTML глубже. Русская опора - читаемый отчет. Если убрать технический шум, то перед нами страница для просмотра результатов человеком. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Командная строка и воспроизводимость' этот термин связан с общей цепочкой анализа. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для

обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину HTML простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть

динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин журнал выполнения в этой книге читается так: по-русски - журнал выполнения. Короткий смысл: последовательность событий, собранных во время запуска. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем журнал выполнения глубже. Русская опора - журнал выполнения. Если убрать технический шум, то перед нами последовательность событий, собранных во время запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Командная строка и воспроизводимость' этот термин связан с общей цепочкой анализа. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для

обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину журнал выполнения простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть

динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Командная строка и воспроизводимость' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова

функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если

есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю почему команда запуска является частью доказательства. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 12. Безопасное чтение байтов

Эта лекция нужна, чтобы разобрать тему 'почему парсер обязан быть недоверчивым' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `byte` в этой книге читается так: по-русски - байт. Короткий смысл: минимальная единица данных в файле. Байт сам по себе не имеет смысла, пока мы не знаем, является ли он частью инструкции, числа, строки, поля PE или служебного выравнивания. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `byte` глубже. Русская опора - байт. Если убрать технический шум, то перед нами минимальная единица данных в файле. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Безопасное чтение байтов' этот термин связан с общей цепочкой анализа. Байт сам по себе не имеет смысла, пока мы не знаем, является ли он частью инструкции, числа, строки, поля PE или служебного выравнивания. Если термин встречается в отчете, его

надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `byte` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин файловое смещение в этой книге читается так: по-русски - файловое смещение. Короткий смысл: позиция байта от начала файла на диске. Это координата hex viewer. Она нужна, когда мы доказываем, что структура действительно читается из файла. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем файловое смещение глубже. Русская опора - файловое смещение. Если убрать технический шум, то перед нами позиция байта от начала файла на диске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Безопасное чтение байтов' этот термин связан с общей цепочкой анализа. Это координата hex viewer. Она нужна, когда мы доказываем, что структура действительно читается из файла. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес

или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину файловое смещение простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин RVA в этой книге читается так: по-русски - относительный виртуальный адрес. Короткий смысл: адрес внутри загруженного образа относительно ImageBase. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем RVA глубже. Русская опора - относительный виртуальный адрес. Если убрать технический шум, то перед нами адрес внутри загруженного образа относительно ImageBase. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Безопасное чтение байтов' этот термин связан с общей цепочкой анализа. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину RVA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин секция в этой книге читается так: по-русски - секция PE. Короткий смысл: именованный диапазон файла и памяти с правами доступа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем секция глубже. Русская опора - секция PE. Если убрать технический шум, то перед нами именованный диапазон файла и памяти с правами доступа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Безопасное чтение байтов' этот термин связан с общей цепочкой анализа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `секция` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин метаданные в этой книге читается так: по-русски - метаданные. Короткий смысл: данные, описывающие другие данные или код. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем метаданные глубже. Русская опора - метаданные. Если убрать технический шум, то перед нами данные, описывающие другие данные или код. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Безопасное чтение байтов' этот термин связан с общей цепочкой анализа. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить

файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину метаданные простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по

адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Безопасное чтение байтов' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова

функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если

есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю почему парсер обязан быть недоверчивым. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 13. PE-парсер как строгий переводчик

Эта лекция нужна, чтобы разобрать тему 'как байты становятся моделью секций и каталогов' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин PE в этой книге читается так: по-русски - Portable Executable. Короткий смысл: формат исполняемых файлов Windows. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем PE глубже. Русская опора - Portable Executable. Если убрать технический шум, то перед нами формат исполняемых файлов Windows. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE-парсер как строгий переводчик' этот термин связан с общей цепочкой анализа. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или

диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `PE` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин секция в этой книге читается так: по-русски - секция PE. Короткий смысл: именованный диапазон файла и памяти с правами доступа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем секция глубже. Русская опора - секция PE. Если убрать технический шум, то перед нами именованный диапазон файла и памяти с правами доступа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE-парсер как строгий переводчик' этот термин связан с общей цепочкой анализа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это

поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `секция` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин RVA в этой книге читается так: по-русски - относительный виртуальный адрес. Короткий смысл: адрес внутри загруженного образа относительно ImageBase. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем RVA глубже. Русская опора - относительный виртуальный адрес. Если убрать технический шум, то перед нами адрес внутри загруженного образа относительно ImageBase. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE-парсер как строгий переводчик' этот термин связан с общей цепочкой анализа. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину RVA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин файловое смещение в этой книге читается так: по-русски - файловое смещение. Короткий смысл: позиция байта от начала файла на диске. Это координата hex viewer. Она нужна, когда мы доказываем, что структура действительно читается из файла. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем файловое смещение глубже. Русская опора - файловое смещение. Если убрать технический шум, то перед нами позиция байта от начала файла на диске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE-парсер как строгий переводчик' этот термин связан с общей цепочкой анализа. Это координата hex viewer. Она нужна, когда мы доказываем, что структура действительно читается из файла. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину файловое смещение простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'PE-парсер как строгий переводчик' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить

файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по

адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'PE-парсер как строгий переводчик' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в

памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть

секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как байты становятся моделью секций и каталогов. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 14. Декодер unwind-метаданных Windows x64

Эта лекция нужна, чтобы разобрать тему 'как анализатор читает RUNTIME_FUNCTION и UNWIND_INFO' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `RUNTIME_FUNCTION` в этой книге читается так: по-русски - запись таблицы функций времени выполнения. Короткий смысл: тройка BeginAddress, EndAddress и UnwindInfoAddress. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `RUNTIME_FUNCTION` глубже. Русская опора - запись таблицы функций времени выполнения. Если убрать технический шум, то перед нами тройка BeginAddress, EndAddress и UnwindInfoAddress. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Декодер unwind-метаданных Windows x64' этот термин связан с общей цепочкой анализа. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Если

термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `RUNTIME_FUNCTION` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `UNWIND_INFO` в этой книге читается так: по-русски - информация раскрутки стека. Короткий смысл: структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `UNWIND_INFO` глубже. Русская опора - информация раскрутки стека. Если убрать технический шум, то перед нами структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Декодер unwind-метаданных Windows x64' этот термин связан с общей цепочкой анализа. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос:

какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `UNWIND_INFO` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин обработчик в этой книге читается так: по-русски - обработчик исключения. Короткий смысл: код или обратный вызов, которому может быть передано управление. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем обработчик глубже. Русская опора - обработчик исключения. Если убрать технический шум, то перед нами код или обратный вызов, которому может быть передано управление. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Декодер unwind-метаданных Windows x64' этот термин связан с общей цепочкой анализа. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Если термин встречается в отчете, его

надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину обработчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин метаданные в этой книге читается так: по-русски - метаданные. Короткий смысл: данные, описывающие другие данные или код. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем метаданные глубже. Русская опора - метаданные. Если убрать технический шум, то перед нами данные, описывающие другие данные или код. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Декодер unwind-метаданных Windows x64' этот термин связан с общей цепочкой анализа. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес

или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину метаданные простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Декодер unwind-метаданных Windows x64' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Декодер unwind-метаданных Windows x64' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как

последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал,

сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как анализатор читает RUNTIME_FUNCTION и UNWIND_INFO. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда

слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 15. Эвристики: почему признак не равен приговору

Эта лекция нужна, чтобы разобрать тему 'как структурные факты превращаются в осторожную оценку' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Эвристики: почему признак не равен приговору' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо

сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Эвристики: почему признак не равен приговору' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин статический анализ в этой книге читается так: по-русски - статический анализ. Короткий смысл: чтение файла без запуска. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем статический анализ глубже. Русская опора - статический анализ. Если убрать технический шум, то перед нами чтение файла без запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Эвристики: почему признак не равен приговору' этот термин связан с общей цепочкой анализа. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `статический анализ` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин метаданные в этой книге читается так: по-русски - метаданные. Короткий смысл: данные, описывающие другие данные или код. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем метаданные глубже. Русская опора - метаданные. Если убрать технический шум, то перед нами данные, описывающие другие данные или код. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Эвристики: почему признак не равен приговору' этот термин связан с общей цепочкой анализа. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину метаданные простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин обработчик в этой книге читается так: по-русски - обработчик исключения. Короткий смысл: код или обратный вызов, которому может быть передано управление. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем обработчик глубже. Русская опора - обработчик исключения. Если убрать технический шум, то перед нами код или обратный вызов, которому может быть передано управление. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Эвристики: почему признак не равен приговору' этот термин связан с общей цепочкой анализа. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что

изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину обработчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Эвристики: почему признак не равен приговору' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как структурные факты превращаются в осторожную оценку. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 16. Динамический сборщик: что реально подтверждает запуск

Эта лекция нужна, чтобы разобрать тему 'как отладчик видит процесс, события, точки останова и срабатывания обработчиков' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин динамическая проверка в этой книге читается так: по-русски - динамическая проверка. Короткий смысл: наблюдение процесса при запуске. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем динамическая проверка глубже. Русская опора - динамическая проверка. Если убрать технический шум, то перед нами наблюдение процесса при запуске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Динамический сборщик: что реально подтверждает запуск' этот термин связан с общей цепочкой анализа.

Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину динамическая проверка простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком,

прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин отладчик в этой книге читается так: по-русски - отладчик. Короткий смысл: инструмент управления процессом и получения события отладки. В динамике отладчик позволяет увидеть exception события, точки останова и срабатывания обработчиков. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем отладчик глубже. Русская опора - отладчик. Если убрать технический шум, то перед нами инструмент управления процессом и получения события отладки. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Динамический сборщик: что реально подтверждает запуск' этот термин связан с общей цепочкой анализа. В динамике отладчик позволяет увидеть exception события, точки

останова и срабатывания обработчиков. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину отладчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к

динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин журнал выполнения в этой книге читается так: по-русски - журнал выполнения. Короткий смысл: последовательность событий, собранных во время запуска. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем журнал выполнения глубже. Русская опора - журнал выполнения. Если убрать технический шум, то перед нами последовательность событий, собранных во время запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Динамический сборщик: что реально подтверждает запуск' этот термин связан с общей цепочкой анализа. Лог нужно читать как эксперимент: что наблюдалось, что не

наблюдалось и какие были ограничения. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину журнал выполнения простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к

динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин обработчик в этой книге читается так: по-русски - обработчик исключения. Короткий смысл: код или обратный вызов, которому может быть передано управление. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем обработчик глубже. Русская опора - обработчик исключения. Если убрать технический шум, то перед нами код или обратный вызов, которому может быть передано управление. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Динамический сборщик: что реально подтверждает запуск' этот термин связан с общей цепочкой анализа.

Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину обработчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком,

прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Динамический сборщик: что реально подтверждает запуск' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести

в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Динамический сборщик: что реально подтверждает запуск' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как отладчик видит процесс, события, точки останова и срабатывания обработчиков. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 17. Отчет как форма доказательства

Эта лекция нужна, чтобы разобрать тему 'почему HTML удобен, а JSON строг' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин HTML в этой книге читается так: по-русски - читаемый отчет. Короткий смысл: страница для просмотра результатов человеком. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем HTML глубже. Русская опора - читаемый отчет. Если убрать технический шум, то перед нами страница для просмотра результатов человеком. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Отчет как форма доказательства' этот термин связан с общей цепочкой анализа. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним

стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину HTML простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `JSON` в этой книге читается так: по-русски - структурированный отчет. Короткий смысл: машиночитаемый формат полей анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `JSON` глубже. Русская опора - структурированный отчет. Если убрать технический шум, то перед нами машиночитаемый формат полей анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Отчет как форма доказательства' этот термин связан с общей цепочкой анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `JSON` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Отчет как форма доказательства' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить

файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по

адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Отчет как форма доказательства' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `Ghidra` в этой книге читается так: по-русски - среда реверс-инжиниринга. Короткий смысл: инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `Ghidra` глубже. Русская опора - среда реверс-инжиниринга. Если убрать технический шум, то перед нами инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Отчет как форма доказательства' этот термин связан с общей цепочкой анализа. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `Ghidra` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Отчет как форма доказательства' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где

возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю почему HTML удобен, а JSON строг. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 18. Нормальные контрольные образцы

Эта лекция нужна, чтобы разобрать тему 'зачем показывать обычные файлы рядом с подозрительными' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `статический анализ` в этой книге читается так: по-русски - статический анализ. Короткий смысл: чтение файла без запуска. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `статический анализ` глубже. Русская опора - статический анализ. Если убрать технический шум, то перед нами чтение файла без запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Нормальные контрольные образцы' этот термин связан с общей цепочкой анализа. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет. Если

термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `статический анализ` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин динамическая проверка в этой книге читается так: по-русски - динамическая проверка. Короткий смысл: наблюдение процесса при запуске. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем динамическая проверка глубже. Русская опора - динамическая проверка. Если убрать технический шум, то перед нами наблюдение процесса при запуске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Нормальные контрольные образцы' этот термин связан с общей цепочкой анализа. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю,

какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину динамическая проверка простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Нормальные контрольные образцы' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Нормальные контрольные образцы' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить

файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по

адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин HTML в этой книге читается так: по-русски - читаемый отчет. Короткий смысл: страница для просмотра результатов человеком. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем HTML глубже. Русская опора - читаемый отчет. Если убрать технический шум, то перед нами страница для просмотра результатов человеком. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Нормальные контрольные образцы' этот термин связан с общей цепочкой анализа. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину HTML простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Нормальные контрольные образцы' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где

возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю зачем показывать обычные файлы рядом с подозрительными. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 19. Подозрительные образцы и ручная проверка

Эта лекция нужна, чтобы разобрать тему 'как построить сильный рассказ по одному бинарнику' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Подозрительные образцы и ручная проверка' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо

сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин RVA в этой книге читается так: по-русски - относительный виртуальный адрес. Короткий смысл: адрес внутри загруженного образа относительно ImageBase. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем RVA глубже. Русская опора - относительный виртуальный адрес. Если убрать технический шум, то перед нами адрес внутри загруженного образа относительно ImageBase. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Подозрительные образцы и ручная проверка' этот термин связан с общей цепочкой анализа. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю,

какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину RVA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `Ghidra` в этой книге читается так: по-русски - среда реверс-инжиниринга. Короткий смысл: инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `Ghidra` глубже. Русская опора - среда реверс-инжиниринга. Если убрать технический шум, то перед нами инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Подозрительные образцы и ручная проверка' этот термин связан с общей цепочкой анализа. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним

стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `Ghidra` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин обработчик в этой книге читается так: по-русски - обработчик исключения. Короткий смысл: код или обратный вызов, которому может быть передано управление. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем обработчик глубже. Русская опора - обработчик исключения. Если убрать технический шум, то перед нами код или обратный вызов, которому может быть передано управление. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Подозрительные образцы и ручная проверка' этот термин связан с общей цепочкой анализа. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Если термин встречается в отчете, его

надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину обработчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин журнал выполнения в этой книге читается так: по-русски - журнал выполнения. Короткий смысл: последовательность событий, собранных во время запуска. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем журнал выполнения глубже. Русская опора - журнал выполнения. Если убрать технический шум, то перед нами последовательность событий, собранных во время запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Подозрительные образцы и ручная проверка' этот термин связан с общей цепочкой анализа. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Если термин встречается в отчете, его надо сразу

перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину журнал выполнения простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Подозрительные образцы и ручная проверка' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком

пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить,

почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как построить сильный рассказ по одному бинарнику. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 20. Задачи с решениями как тренировка мышления

Эта лекция нужна, чтобы разобрать тему 'как переходить от вопроса к проверке без скачков' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин RVA в этой книге читается так: по-русски - относительный виртуальный адрес. Короткий смысл: адрес внутри загруженного образа относительно ImageBase. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем RVA глубже. Русская опора - относительный виртуальный адрес. Если убрать технический шум, то перед нами адрес внутри загруженного образа относительно ImageBase. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Задачи с решениями как тренировка мышления' этот термин связан с общей цепочкой анализа. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Если термин встречается в отчете, его надо сразу

перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину RVA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин секция в этой книге читается так: по-русски - секция PE. Короткий смысл: именованный диапазон файла и памяти с правами доступа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем секция глубже. Русская опора - секция PE. Если убрать технический шум, то перед нами именованный диапазон файла и памяти с правами доступа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Задачи с решениями как тренировка мышления' этот термин связан с общей цепочкой анализа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле

отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `секция` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `UNWIND_INFO` в этой книге читается так: по-русски - информация раскрутки стека. Короткий смысл: структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `UNWIND_INFO` глубже. Русская опора - информация раскрутки стека. Если убрать технический шум, то перед нами структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Задачи с решениями как тренировка мышления' этот термин связан с общей цепочкой анализа. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним

стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `UNWIND_INFO` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `JSON` в этой книге читается так: по-русски - структурированный отчет. Короткий смысл: машиночитаемый формат полей анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `JSON` глубже. Русская опора - структурированный отчет. Если убрать технический шум, то перед нами машиночитаемый формат полей анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Задачи с решениями как тренировка мышления' этот термин связан с общей цепочкой анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `JSON` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `Ghidra` в этой книге читается так: по-русски - среда реверс-инжиниринга. Короткий смысл: инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `Ghidra` глубже. Русская опора - среда реверс-инжиниринга. Если убрать технический шум, то перед нами инструмент для дизассемблерное представление, decompiler, карта памяти и ручной проверки. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Задачи с решениями как тренировка мышления' этот термин связан с общей цепочкой анализа. Ghidra нужна не как оракул, а как микроскоп для проверки конкретного RVA. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `Ghidra` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Задачи с решениями как тренировка мышления' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как

последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал,

сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как переходить от вопроса к проверке без скачков. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда

слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 21. Глоссарий как карта, а не список слов

Эта лекция нужна, чтобы разобрать тему 'почему термин надо связывать с действием' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `byte` в этой книге читается так: по-русски - байт. Короткий смысл: минимальная единица данных в файле. Байт сам по себе не имеет смысла, пока мы не знаем, является ли он частью инструкции, числа, строки, поля PE или служебного выравнивания. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `byte` глубже. Русская опора - байт. Если убрать технический шум, то перед нами минимальная единица данных в файле. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Глоссарий как карта, а не список слов' этот термин связан с общей цепочкой анализа. Байт сам по себе не имеет смысла, пока мы не знаем, является ли он частью инструкции, числа, строки, поля PE или служебного выравнивания. Если термин встречается в

отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `byte` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин RVA в этой книге читается так: по-русски - относительный виртуальный адрес. Короткий смысл: адрес внутри загруженного образа относительно ImageBase. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем RVA глубже. Русская опора - относительный виртуальный адрес. Если убрать технический шум, то перед нами адрес внутри загруженного образа относительно ImageBase. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Глоссарий как карта, а не список слов' этот термин связан с общей цепочкой анализа. RVA связывает PE-таблицы с памятью, но еще не является реальным адресом процесса. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или

диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину RVA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин VA в этой книге читается так: по-русски - виртуальный адрес. Короткий смысл: адрес в памяти конкретного процесса. VA получается из ImageBase и RVA; именно такие адреса часто видны в отладчик и Ghidra. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем VA глубже. Русская опора - виртуальный адрес. Если убрать технический шум, то перед нами адрес в памяти конкретного процесса. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Глоссарий как карта, а не список слов' этот термин связан с общей цепочкой анализа. VA получается из ImageBase и RVA; именно такие адреса часто видны в отладчик и Ghidra. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину VA простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин PE в этой книге читается так: по-русски - Portable Executable. Короткий смысл: формат исполняемых файлов Windows. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем PE глубже. Русская опора - Portable Executable. Если убрать технический шум, то перед нами формат исполняемых файлов Windows. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Глоссарий как карта, а не список слов' этот термин связан с общей цепочкой анализа. PE задает договор между файлом, компоновщиком, загрузчиком Windows и инструментом анализа. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для

обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину PE простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть

динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Глоссарий как карта, а не список слов' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Глоссарий как карта, а не список слов' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где

возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю почему термин надо связывать с действием. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 22. Свидетельства и покрытие наблюдения

Эта лекция нужна, чтобы разобрать тему 'как понять силу и неполноту наблюдений' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Свидетельства и покрытие наблюдения' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или

диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин динамическая проверка в этой книге читается так: по-русски - динамическая проверка. Короткий смысл: наблюдение процесса при запуске. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем динамическая проверка глубже. Русская опора - динамическая проверка. Если убрать технический шум, то перед нами наблюдение процесса при запуске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Свидетельства и покрытие наблюдения' этот термин связан с общей цепочкой анализа. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это

поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину динамическая проверка простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин журнал выполнения в этой книге читается так: по-русски - журнал выполнения. Короткий смысл: последовательность событий, собранных во время запуска. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем журнал выполнения глубже. Русская опора - журнал выполнения. Если убрать технический шум, то перед нами последовательность событий, собранных во время запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Свидетельства и покрытие наблюдения' этот термин связан с общей цепочкой анализа. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю,

какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину журнал выполнения простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Свидетельства и покрытие наблюдения' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если

это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `JSON` в этой книге читается так: по-русски - структурированный отчет. Короткий смысл: машиночитаемый формат полей анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `JSON` глубже. Русская опора - структурированный отчет. Если убрать технический шум, то перед нами машиночитаемый формат полей анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Свидетельства и покрытие наблюдения' этот термин связан с общей цепочкой анализа. JSON помогает проверить точные поля без влияния HTML-верстки. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `JSON` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Свидетельства и покрытие наблюдения' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как

последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал,

сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как понять силу и неполноту наблюдений. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть

шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 23. Система 27 признаков

Эта лекция нужна, чтобы разобрать тему 'как признаки вырастают из PE, unwind метаданные и поведение во время выполнения' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Система 27 признаков' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский

инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину признак простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `RUNTIME_FUNCTION` в этой книге читается так: по-русски - запись таблицы функций времени выполнения. Короткий смысл: тройка BeginAddress, EndAddress и UnwindInfoAddress. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `RUNTIME_FUNCTION` глубже. Русская опора - запись таблицы функций времени выполнения. Если убрать технический шум, то перед нами тройка BeginAddress, EndAddress и UnwindInfoAddress. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Система 27 признаков' этот термин связан с общей цепочкой анализа. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что

изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `RUNTIME_FUNCTION` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `UNWIND_INFO` в этой книге читается так: по-русски - информация раскрутки стека. Короткий смысл: структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `UNWIND_INFO` глубже. Русская опора - информация раскрутки стека. Если убрать технический шум, то перед нами структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Система 27 признаков' этот термин связан с общей цепочкой анализа. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что

изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `UNWIND_INFO` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин секция в этой книге читается так: по-русски - секция PE. Короткий смысл: именованный диапазон файла и памяти с правами доступа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем секция глубже. Русская опора - секция PE. Если убрать технический шум, то перед нами именованный диапазон файла и памяти с правами доступа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Система 27 признаков' этот термин связан с общей цепочкой анализа. Секция объясняет, почему одни байты исполняются, другие читаются как данные, а третьи выглядят необычно. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это

поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `секция` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин обработчик в этой книге читается так: по-русски - обработчик исключения. Короткий смысл: код или обратный вызов, которому может быть передано управление. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем обработчик глубже. Русская опора - обработчик исключения. Если убрать технический шум, то перед нами код или обратный вызов, которому может быть передано управление. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Система 27 признаков' этот термин связан с общей цепочкой анализа. Обработчик надо проверять как адрес кода: где он лежит, исполняема ли секция, достигался ли он динамически. Если термин встречается в отчете, его надо сразу перевести в

русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину обработчик простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Система 27 признаков' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как признаки вырастают из PE, unwind метаданные и поведение во время выполнения. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 24. Точность, валидация и эксперимент

Эта лекция нужна, чтобы разобрать тему 'почему один запуск не является всей истиной' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин динамическая проверка в этой книге читается так: по-русски - динамическая проверка. Короткий смысл: наблюдение процесса при запуске. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем динамическая проверка глубже. Русская опора - динамическая проверка. Если убрать технический шум, то перед нами наблюдение процесса при запуске. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Точность, валидация и эксперимент' этот термин связан с общей цепочкой анализа. Она дает время выполнения-факты, но зависит от аргументов, тайм-аут, окружения и покрытого пути. Если термин встречается в отчете, его надо сразу

перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину динамическая проверка простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Точность, валидация и эксперимент' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин журнал выполнения в этой книге читается так: по-русски - журнал выполнения. Короткий смысл: последовательность событий, собранных во время запуска. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем журнал выполнения глубже. Русская опора - журнал выполнения. Если убрать технический шум, то перед нами последовательность событий, собранных во время запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Точность, валидация и эксперимент' этот термин связан с общей цепочкой анализа. Лог нужно читать как эксперимент: что наблюдалось, что не наблюдалось и какие были ограничения. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину журнал выполнения простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин статический анализ в этой книге читается так: по-русски - статический анализ. Короткий смысл: чтение файла без запуска. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем статический анализ глубже. Русская опора - статический анализ. Если убрать технический шум, то перед нами чтение файла без запуска. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Точность, валидация и эксперимент' этот термин связан с общей цепочкой анализа. Он силен структурной полнотой, но не доказывает, какой путь выполнения реально произойдет. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить

файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `статический анализ` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по

адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин HTML в этой книге читается так: по-русски - читаемый отчет. Короткий смысл: страница для просмотра результатов человеком. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем HTML глубже. Русская опора - читаемый отчет. Если убрать технический шум, то перед нами страница для просмотра результатов человеком. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Точность, валидация и эксперимент' этот термин связан с общей цепочкой анализа. HTML удобен для демонстрации, но доказательство надо уметь сверять с JSON. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину HTML простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Точность, валидация и эксперимент' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где

возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал, сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю почему один запуск не является всей истиной. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том III. Лекция 25. Совместимость и будущие версии unwind метаданные

Эта лекция нужна, чтобы разобрать тему 'как поддерживать новые форматы без ложных выводов' не как перечень английских названий, а как связный участок инженерной картины. Сначала мы говорим простым языком, затем переходим к строгим структурам Windows и только после этого связываем тему с EH/Unwind Analyzer.

Главный принцип такой: любой низкоуровневый термин должен иметь три опоры. Первая - человеческий смысл, то есть зачем эта вещь существует. Вторая - точная техническая форма, то есть поле, структура, адрес или событие. Третья - проверка, которую можно выполнить в отчете, JSON, Ghidra или отладчик. Если хотя бы одной опоры нет, термин остается пустым английским словом.

Термины лекции простыми словами

Термин `UNWIND_INFO` в этой книге читается так: по-русски - информация раскрутки стека. Короткий смысл: структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Она объясняет системе, как безопасно вернуться к вызывающему кадру. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `UNWIND_INFO` глубже. Русская опора - информация раскрутки стека. Если убрать технический шум, то перед нами структура Windows x64 с описанием prolog, stack operations, flags и обработчик. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Совместимость и будущие версии unwind метаданные' этот термин связан с общей цепочкой анализа. Она объясняет системе, как безопасно вернуться к вызывающему кадру.

Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `UNWIND_INFO` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с

конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин метаданные в этой книге читается так: по-русски - метаданные. Короткий смысл: данные, описывающие другие данные или код. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем метаданные глубже. Русская опора - метаданные. Если убрать технический шум, то перед нами данные, описывающие другие данные или код. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Совместимость и будущие версии unwind метаданные' этот термин связан с общей цепочкой анализа. В EH/Unwind метаданные управляют тем, как Windows понимает функции и раскручивает стек. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле

отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину метаданные простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин `RUNTIME_FUNCTION` в этой книге читается так: по-русски - запись таблицы функций времени выполнения. Короткий смысл: тройка BeginAddress, EndAddress и UnwindInfoAddress. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем `RUNTIME_FUNCTION` глубже. Русская опора - запись таблицы функций времени выполнения. Если убрать технический шум, то перед нами тройка BeginAddress, EndAddress и UnwindInfoAddress. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Совместимость и будущие версии unwind метаданные' этот термин связан с общей цепочкой анализа. Эта запись говорит Windows, где находится функция и где лежит ее карта раскрутки. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю,

какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `RUNTIME_FUNCTION` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин признак в этой книге читается так: по-русски - признак. Короткий смысл: названный проверяемый факт анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем признак глубже. Русская опора - признак. Если убрать технический шум, то перед нами названный проверяемый факт анализа. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Совместимость и будущие версии unwind метаданные' этот термин связан с общей цепочкой анализа. Признак не является приговором; он должен иметь адрес, правило, свидетельства и ограничение вывода. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что

изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `признак` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Термин уверенность в этой книге читается так: по-русски - уверенность. Короткий смысл: инженерная оценка силы свидетельств. Это не процент вредоносности, а мера согласованности найденных фактов. Важно не просто запомнить английское слово, а сразу привязать его к действию: где это поле видно в отчете, каким адресом оно выражено и как его проверить руками.

Разберем уверенность глубже. Русская опора - уверенность. Если убрать технический шум, то перед нами инженерная оценка силы свидетельств. Но простого перевода мало: термин должен отвечать на вопрос, где объект находится, кто его создал, кто его использует и как его проверить.

В рамках лекции 'Совместимость и будущие версии unwind метаданные' этот термин связан с общей цепочкой анализа. Это не процент вредоносности, а мера согласованности найденных фактов. Если термин встречается в отчете, его надо сразу перевести в русский инженерный вопрос: какое поле отчета я читаю, какой адрес или диапазон за ним стоит, и что изменится, если это поле неверно.

Строгая привычка такая: сначала назвать тип объекта, потом систему координат, потом владельца. Для адреса нужно уточнить файловое смещение, RVA или VA. Для секции - права и диапазон. Для обработчика - исполняемость адреса и факт достижения во время запуска.

Связь с EH/Unwind Analyzer проявляется в том, что проект не работает с абстрактными словами. Он читает байты, строит модель PE, переводит адреса, декодирует unwind-метаданные, создает признак и показывает человеку доказуемые детали. Поэтому объяснение термина должно заканчиваться ссылкой на таблицу отчета, JSON-поле, адрес в Ghidra или строку динамического журнала.

Проверка понимания по термину `уверенность` простая: найди один пример в отчете, перепиши его по-русски и добавь фразу 'я могу подтвердить это через ...'. Если вместо многоточия получается назвать конкретный HTML-блок, JSON-поле, Ghidra-адрес или строку журнала выполнения, термин стал рабочим знанием.

Теперь свяжем это со слоями. В файле на диске объект существует как байты и смещения. В загруженном образе он получает место в карте PE и может выражаться через RVA. В процессе он может проявиться как виртуальный адрес, событие отладки или срабатывание обработчика. Один термин может проходить через все эти слои, и именно поэтому его нельзя читать как отдельное слово.

Для Windows важен не красивый перевод, а соблюдение договора. Если объект относится к PE, он должен согласовываться с заголовками, секциями и каталогами данных. Если он относится к x64 unwind, он должен согласовываться с диапазоном функции, стеком, прологом и правилами обработки исключений. Если он относится к динамике, его надо читать как наблюдение конкретного запуска с конкретными ограничениями.

Маршрут ручной проверки всегда похож на маленькое расследование. Сначала найди объект в HTML, чтобы увидеть

человеческое объяснение. Затем открой JSON, чтобы проверить точные поля без влияния верстки. После этого перейди в Ghidra по адресу или секции и посмотри, что находится вокруг. Если есть динамический отчет, сравни его журнал с тем, что предполагала статика.

Типичная ошибка выглядит так: человек узнает термин, но пропускает вопрос о границах. Граница может быть файловой, адресной, секционной, временной или доказательной. Файловая граница отвечает на вопрос, есть ли нужные байты. Адресная - правильно ли переведена координата. Секционная - какие права и роль у области. Временная - проявилось ли это при запуске. Доказательная - можно ли на основании этого делать сильный вывод.

Как эта тема работает внутри Windows и PE

В теме 'Совместимость и будущие версии unwind метаданные' важно видеть не отдельные детали, а цепочку. Файл на диске содержит байты. PE-заголовки объясняют, какие байты станут секциями. Секции получают виртуальные адреса и права. На x64 часть кода связывается с таблицами `RUNTIME_FUNCTION`, а эти таблицы ведут к `UNWIND_INFO`. Когда происходит исключение или отладчик ловит событие, Windows опирается не на догадку, а на эти формальные описания. Поэтому любая аномалия в таблицах, адресах или размещении метаданные является поводом для внимательного анализа.

Новичку легко потеряться, потому что одни и те же числа путешествуют между слоями. Например, значение может быть RVA в таблице, VA в Ghidra, файловое смещение в hex viewer и строкой в JSON. Глубокое понимание начинается там, где ты перестаешь говорить просто 'адрес' и каждый раз уточняешь, в каком пространстве координат находишься.

Чтобы это стало не теорией ради теории, представь один полный путь. Сначала файл лежит на диске, и мы можем открыть его как

последовательность байтов. Затем PE-заголовки говорят, какие области станут секциями. Затем загрузчик Windows создает image в памяти. Затем процессор исполняет код, а ABI задает правила вызова функций. Затем при исключении Windows ищет запись таблицы функций и читает unwind метаданные. Analyzer идет по этой же цепочке снаружи и проверяет, где она выглядит согласованной, а где возникает разрыв.

Такой путь важен еще и потому, что он защищает от ложных выводов. Если видеть только последнюю карточку verdict, можно решить, что инструмент просто выдает оценку. Но внутри оценки лежит много маленьких проверок: границы секций, отображение RVA, корректность структуры, права памяти, наличие обработчик, события времени выполнения и ограничения эксперимента. Чем больше этих проверок ты можешь объяснить словами, тем меньше книга похожа на список терминов.

Как это проявляется в EH/Unwind Analyzer

В проекте эта тема проходит через несколько этапов. Сначала парсер читает PE и проверяет, можно ли доверять размерам, секциям и каталогам. Потом unwind декодер пытается прочитать таблицы функций и метаданные. Затем heuristics engine называет проверяемые факты признаками. Если включена динамика, collector запускает процесс под наблюдением и собирает время выполнения-события. Наконец report writer показывает это человеку в HTML и JSON.

Смысл такого pipeline не в том, чтобы заменить человека. Он нужен, чтобы быстро поднять факты, которые человек затем проверяет. Analyzer может сказать: 'вот признак, вот affected count, вот RVA, вот секция, вот журнал выполнения'. Человек должен объяснить, почему это важно и где границы вывода.

Поэтому в учебном чтении проекта всегда держи две колонки. В первой колонке - что сделал инструмент: прочитал, декодировал,

сравнил, зарегистрировал событие, сформировал отчет. Во второй колонке - что должен сделать человек: проверить адрес, открыть секцию, сравнить Ghidra, прочитать JSON, назвать ограничение. Если есть только первая колонка, получается черный ящик. Если есть только вторая, теряется автоматизация. Сила проекта в их соединении.

Типичная ошибка и правильное рассуждение

Типичная ошибка - увидеть английское имя поля и начать повторять его как заклинание. Например, сказать 'UNWIND_INFO неправильный' недостаточно. Нужно сказать: какая функция ссылается на эту метаданные, какой RVA используется, в какую секцию он попадает, что именно не декодируется или выглядит необычно, и можно ли подтвердить это динамически.

Правильное рассуждение идет медленнее, но сильнее. Сначала объект: таблица, секция, обработчик или событие. Потом координата: файловое смещение, RVA или VA. Потом правило: что ожидает PE/Windows x64. Потом наблюдение: что именно нарушено. Потом проверка: где это видно в отчете и Ghidra. В конце ограничение: чего этот факт еще не доказывает.

Мини-лекция для защиты

Если эту тему нужно объяснить преподавателю за одну минуту, начни не с английского термина, а с роли. Скажи: 'В этой части проекта я проверяю как поддерживать новые форматы без ложных выводов. Это важно, потому что Windows должна понимать, где находится функция, как восстановить стек и куда передавать управление при исключении. Если структура или адреса выглядят необычно, анализатор фиксирует не приговор, а проверяемый признак'.

Затем покажи один адрес и один отчет. Не надо показывать всю книгу. Достаточно одной цепочки: HTML summary -> Static признаки -> Функции времени выполнения -> JSON field -> Ghidra address. Когда

слушатель видит эту цепочку, английские слова перестают быть шумом и становятся инструментами.

Самопроверка

Ответь письменно на три вопроса. Что в этой теме является объектом анализа? В какой системе координат он записан? Какая ручная проверка подтверждает, что анализатор не выдумал вывод? Если ответы получились конкретными, лекция усвоена. Если ответы общие, вернись к терминам и найди адрес, секцию или событие.

Том IV. Почему 27 признаков имеют смысл: теория, доказательство и осторожность вывода

Признак в этом проекте - не приговор и не ярлык "malware". Это аккуратно названный низкоуровневый факт. Хорошая инженерная культура начинается с того, что мы не преувеличиваем смысл факта. Мы говорим: вот что наблюдается; вот в какой семье свидетельства это находится; вот почему оно необычно; вот как проверить руками; вот какие ограничения у вывода.

Эта глава связывает признаки с теорией. Она не заменяет отдельный учебник по признакам, а показывает, почему признаки вообще вырастают из PE, секций, RUNTIME_FUNCTION, UNWIND_INFO, обработчики и dynamic свидетельства.

1. pe_section_raw_oob

Что наблюдается: секция заявляет raw range за пределами файла. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть pe_section_raw_oob, сначала найди affected count, representative RVA и message.

Теоретическая опора: PE секция header должен указывать на реально существующие байты. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: сравнить PointerToRawData + SizeOfRawData с размером файла. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю

нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

2. pe_section_virtual_oob

Что наблюдается: виртуальный диапазон секции выходит за image. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть pe_section_virtual_oob, сначала найди affected count, representative RVA и message.

Теоретическая опора: loader строит image по SectionAlignment и SizeOfImage. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: сравнить VirtualAddress + VirtualSize с SizeOfImage. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

3. pe_section_raw_overlap

Что наблюдается: две секции пересекаются в raw file. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть pe_section_raw_overlap, сначала найди affected count, representative RVA и message.

Теоретическая опора: один байт файла не должен неоднозначно принадлежать двум секциям. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется:

целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: отсортировать raw ranges и найти пересечение. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

4. pe_section_virtual_overlap

Что наблюдается: две секции пересекаются в виртуальном image. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть pe_section_virtual_overlap, сначала найди affected count, representative RVA и message.

Теоретическая опора: RVA должен иметь однозначный секция context. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: отсортировать virtual ranges и найти пересечение. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

5. pe_section_misaligned

Что наблюдается: секция нарушает ожидаемое выравнивание. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть pe_section_misaligned, сначала найди affected count, representative RVA и message.

Теоретическая опора: FileAlignment и SectionAlignment задают сетку размещения. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: проверить адреса и размеры секций на кратность alignment. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

6. pe_exception_directory_oob

Что наблюдается: exception directory выходит за допустимые границы. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть pe_exception_directory_oob, сначала найди affected count, representative RVA и message.

Теоретическая опора: каталог исключений должен быть mappable в секции/image. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: перевести directory RVA в файловое смещение и проверить size. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

7. no_runtime_functions

Что наблюдается: в x64 image нет время выполнения function entries. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть no_runtime_functions, сначала найди affected count, representative RVA и message.

Теоретическая опора: non-leaf x64 code обычно требует unwind метаданные. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: сначала понять, есть ли исполняемый non-leaf код. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

8. invalid_ranges

Что наблюдается: BeginAddress и EndAddress образуют невозможные диапазоны. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть invalid_ranges, сначала найди affected count, representative RVA и message.

Теоретическая опора: диапазон функции должен иметь begin < end. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: найти строки с нулевым или отрицательным размером. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

9. rf_oob

Что наблюдается: время выполнения function range выходит за image или code context. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть rf_oob, сначала найди affected count, representative RVA и message.

Теоретическая опора: функция должна ссылаться на существующий код. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: сравнить begin/end с секциями и SizeOfImage. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

10. unwind_rva_oob

Что наблюдается: UnwindInfoAddress не указывает на валидное место. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть unwind_rva_oob, сначала найди affected count, representative RVA и message.

Теоретическая опора: RUNTIME_FUNCTION должен вести к читаемой UNWIND_INFO. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: проверить Unwind RVA через секция mapping. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

11. rf_unsorted

Что наблюдается: таблица функций не отсортирована по begin. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть rf_unsorted, сначала найди affected count, representative RVA и message.

Теоретическая опора: Windows ожидает упорядоченный поиск function entries. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: отсортировать begin values и найти нарушение. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

12. rf_overlaps

Что наблюдается: диапазоны функций пересекаются. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть rf_overlaps, сначала найди affected count, representative RVA и message.

Теоретическая опора: один адрес кода не должен принадлежать двум разным function entries. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: сравнить соседние begin/end после сортировки. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

13. duplicate_begin

Что наблюдается: несколько записей начинаются с одного адреса. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть duplicate_begin, сначала найди affected count, representative RVA и message.

Теоретическая опора: BeginAddress должен идентифицировать диапазон. Семья свидетельства: `table_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: найти одинаковые BeginAddress. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

14. unwind_parse_fail

Что наблюдается: UNWIND_INFO не удалось корректно разобрать. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть unwind_parse_fail, сначала найди affected count, representative RVA и message.

Теоретическая опора: layout структуры должен помещаться в доступные байты. Семья свидетельства: `unwind_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: открыть bytes UNWIND_INFO и проверить header/codes/tail. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

15. unwind_anomalies

Что наблюдается: UNWIND_INFO разобран, но содержит необычные свойства. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть unwind_anomalies, сначала найди affected count, representative RVA и message.

Теоретическая опора: unwind codes должны согласовываться с prolog/frame. Семья свидетельства: `unwind_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: сравнить unwind operations с дизассемблерное представление prolog. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

16. unwind_reuse_moderate

Что наблюдается: несколько функций повторно используют один UnwindInfoRVA. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть unwind_reuse_moderate, сначала найди affected count, representative RVA и message.

Теоретическая опора: reuse бывает нормальным, но требует контекста. Семья свидетельства: `unwind_reuse`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: проверить, насколько похожи функции и есть ли другие signals. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

17. unwind_reuse_strong

Что наблюдается: слишком много функций используют одну unwind запись. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть unwind_reuse_strong, сначала найди affected count, representative RVA и message.

Теоретическая опора: массовая централизация метаданные может скрывать pattern. Семья свидетельства: `unwind_reuse`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: посчитать долю функций с одинаковым UnwindInfoRVA. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

18. handler_dominance

Что наблюдается: один обработчик обслуживает слишком много функций. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть handler_dominance, сначала найди affected count, representative RVA и message.

Теоретическая опора: обработчик может быть время выполнения helper или центральный dispatcher. Семья свидетельства: `handler_dominance`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: перейти к обработчик и оценить его роль. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

19. unwind_in_exec_section

Что наблюдается: unwind метаданные лежит в executable секция. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть unwind_in_exec_section, сначала найди affected count, representative RVA и message.

Теоретическая опора: метаданные обычно read-only data, а не код. Семья свидетельства: `metadata_placement`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: проверить секция permissions для Unwind RVA. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

20. unwind_unusual_section

Что наблюдается: unwind метаданные лежит в необычной секции. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть unwind_unusual_section, сначала найди affected count, representative RVA и message.

Теоретическая опора: секция .xdata ожидаема, но .weird требует объяснения. Семья свидетельства: `metadata_placement`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: сравнить секция name с нормальными PE conventions. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

21. unwind_writable_section

Что наблюдается: unwind метаданные лежит в writable секция. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть unwind_writable_section, сначала найди affected count, representative RVA и message.

Теоретическая опора: изменяемая метаданные опаснее неизменяемой. Семья свидетельства: `metadata_placement`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: проверить W permission у секции. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

22. handler_nonexec_section

Что наблюдается: обработчик указывает в неисполняемую область. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть handler_nonexec_section, сначала найди affected count, representative RVA и message.

Теоретическая опора: обработчик должен быть кодом. Семья свидетельства: `metadata_placement`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: сопоставить обработчик RVA с секция permissions. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

23. handler_writable_section

Что наблюдается: обработчик находится в writable секция. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть handler_writable_section, сначала найди affected count, representative RVA и message.

Теоретическая опора: исполняемый код в writable области повышает риск. Семья свидетельства: `metadata_placement`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: найти секция для обработчик целевая программа. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

24. placement_exec_nonexec_handler_combo

Что наблюдается: сочетание подозрительного placement и обработчик context. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть placement_exec_nonexec_handler_combo, сначала найди affected count, representative RVA и message.

Теоретическая опора: комбинации важнее одиночного факта. Семья свидетельства: `metadata_placement`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: проверить обе стороны: метаданные и обработчик. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

25. placement_integrity_combo

Что наблюдается: placement подтверждается structural integrity проблемой. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть placement_integrity_combo, сначала найди affected count, representative RVA и message.

Теоретическая опора: сильный case появляется при согласовании разных слоев. Семья свидетельства: `unwind_integrity`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: связать секция anomaly с parse/range anomaly. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

26. reuse_dominance_combo

Что наблюдается: reuse метаданные связан с доминирующим обработчик. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть reuse_dominance_combo, сначала найди affected count, representative RVA и message.

Теоретическая опора: централизация двух разных объектов усиливает гипотезу. Семья свидетельства: `unwind_reuse`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: посчитать reuse и dominance в одном отчете. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

27. writable_metadata_handler_combo

Что наблюдается: writable метаданные и writable обработчик усиливают друг друга. Это нужно читать как техническое утверждение, а не эмоциональный вывод. Если в отчете есть writable_metadata_handler_combo, сначала найди affected count, representative RVA и message.

Теоретическая опора: изменяемость данных и кода создает сильный сигнал. Семья свидетельства: `metadata_placement`. Семья показывает, какой слой проверяется: целостность таблиц, декодирование unwind метаданные, placement, reuse или обработчик model.

Как проверять: проверить permissions для обоих RVA. После этого желательно открыть HTML/JSON и Ghidra. В HTML удобно увидеть итоговую таблицу, в JSON - точные поля, в Ghidra - код и контекст.

Как говорить на защите: 'анализатор обнаружил проверяемый структурный факт; я показываю адрес или диапазон, объясняю нарушенный инвариант, затем подтверждаю или ослабляю вывод ручной проверкой'. Такая формулировка честнее и сильнее, чем резкое заявление без доказательства.

Том V. Учебный маршрут: как из терминов собрать профессию

Эта глава добавлена не для объема, а для связности. В длинной технической теме легко выучить отдельные слова и все равно не понимать, что делать перед реальным бинарником. Здесь каждый модуль устроен как маленькая лекция: сначала человеческий смысл, потом строгая связь с Windows и PE, потом типичная ошибка, потом упражнение, которое можно выполнить руками.

Маршрут намеренно идет от мышления к проверке. Реверс-инжиниринг начинается не с инструмента, а с дисциплины вопроса. Что я сейчас вижу: байты, адрес, секцию, функцию, обработчик, событие отладчик или итоговую оценку? Какой слой создал это значение? Какой другой инструмент может подтвердить или ослабить вывод? Если эти вопросы становятся привычкой, Ghidra, Microsoft documentation и отчеты проекта перестают быть разрозненными окнами и складываются в одну систему.

Читай эту главу как набор лекций перед практикой. После каждого модуля лучше открыть один реальный отчет из `reports`, один образец из набора комбинированный и один исходный файл проекта. Тогда теория сразу получает опору в конкретных адресах, таблицах и командах.

Модуль 1. Как думать о бинарнике до открытия Ghidra

Новичок часто хочет сразу нажать Analyze и получить ответ. В реверс-инжиниринге это почти всегда слишком быстро. Сначала нужно принять простую мысль: бинарник не является текстом программы, который кто-то аккуратно подписал для человека. Это набор байтов, созданный компилятором, компоновщиком и иногда вручную измененный человеком или инструментом. Поэтому первая работа аналитика - не угадать смысл, а восстановить уровни представления.

В этой теме полезно разделить четыре слоя: файл на диске, образ после загрузки, исполняемый процесс и аналитический отчет. Файл живет в координатах файловое смещение. Загруженный образ живет в RVA и VA. Процесс добавляет реальные адреса, потоки, исключения и API-вызовы. Отчет анализатор собирает эти наблюдения и превращает их в проверяемые утверждения.

Типичная ошибка - говорить 'по адресу 0x7100 лежит плохой код', не уточнив, это файловое смещение, RVA или VA. Для защиты проекта такая неточность опасна: оппонент может попросить показать этот адрес в Ghidra или hex viewer, и без слоя координат ответ рассыпается.

Практическая проверка: возьми один отчет, выпиши из него ImageBase, representative RVA и секцию. Затем отдельно покажи, как этот RVA превращается в VA при запуске и как он превращается в файловое смещение при чтении байтов из файла.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 2. Почему PE - это договор, а не просто формат

Документация Microsoft описывает PE/COFF как структуру исполняемых файлов и объектных файлов Windows. Но для учебника важнее увидеть смысл: PE - это договор между тем, кто собрал файл, и тем, кто будет его загружать. В договоре сказано, где заголовки, где таблица секций, где каталоги данных, какие области нужно отобразить в память и какие права у этих областей.

Когда анализатор читает PE, он ведет себя как строгий проверяющий договора. Он не запускает программу и не верит ей на слово. Он смотрит, можно ли найти PE signature, согласуются ли размеры, попадают ли директории в образ, можно ли перевести RVA в файловое смещение, не указывает ли таблица на область, которой в файле нет.

Связь с проектом прямая: без PE парсер все последующие проверки бессмысленны. Нельзя декодировать UNWIND_INFO, пока не доказано, что адрес этой структуры действительно отображается в байты файла. Нельзя уверенно говорить о секции, пока таблица секций не прочитана и не проверена.

Практическая проверка: в HTML-отчете открой блок Sections и найди .text, .pdata, .xdata или необычную секцию. Для каждой запиши Virtual address, Virtual size, Raw offset, Raw size и Permissions. После этого объясни, какая часть существует на диске, а какая появляется как память процесса.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 3. Как не перепутать файловое смещение, RVA и VA

Три адреса - главный экзамен на зрелость в этой теме. Файловое смещение отвечает на вопрос 'где байт лежит в файле'. RVA отвечает на вопрос 'где объект находится внутри образа относительно ImageBase'. VA отвечает на вопрос 'где объект оказался в адресном пространстве конкретного процесса'. Это не три названия одного и того же, а три разные системы координат.

Microsoft PE Format отдельно определяет RVA и VA именно потому, что файл на диске и память процесса устроены по-разному. Секция может иметь один размер на диске и другой размер в памяти. Между секциями могут быть выравнивания. ASLR может изменить фактическую базу загрузки. Поэтому прямое сравнение offset и адрес во время выполнения без перевода - ошибка.

В EH/Unwind Analyzer это особенно важно для RUNTIME_FUNCTION. BeginAddress, EndAddress и UnwindInfoAddress являются image-relative значениями. Отчет показывает их как RVA, а динамический лог может показывать VA. Если увидеть только числа и не понять слой, можно решить, что статический и динамический анализ расходятся, хотя они говорят об одном объекте.

Практическая проверка: выбери функцию из таблицы Функции времени выполнения. Вычисли VA = loaded base + Begin RVA. Потом найди секцию, куда попадает Begin RVA, и вычисли файловое смещение = PointerToRawData + (RVA - VirtualAddress). Если все три точки совпали в отчете, Ghidra и hex viewer, координаты освоены.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 4. Секции и права как язык намерений

Секция PE - это не папка и не красивое имя. Это диапазон байтов и памяти с ролью, размерами и правами. Обычно .text исполняется и читается, .rdata читается, .data читается и пишется, .pdata содержит таблица функций времени выполнения, .xdata содержит unwind метаданные. Такие имена не являются магией, но они образуют культурную норму Windows x64.

Аномалия начинается там, где структура нарушает ожидание. Если unwind метаданные оказывается в writable секция, это не автоматически вредоносность, но это повод спросить: почему данные, от которых зависит раскрутка стека и обработка исключений, находятся там, где программа может их менять? Если секция с метаданные называется странно, это снова вопрос к намерению и устойчивости.

Проект не должен кричать 'malware' только потому, что секция необычна. Он должен сказать точнее: unwind секция unusual, unwind секция writable, code секция unusual, обработчик outside image или другой конкретный факт. Сила отчета не в громкости вывода, а в проверяемости наблюдения.

Практическая проверка: найди строку с Anomalies/error в Функции времени выполнения. Если там написано unwind секция unusual или writable, перейди в таблицу Sections и покажи, что именно в правах или имени секции вызвало признак.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 5. x64 ABI как правила дорожного движения для функций

Calling convention, или соглашение о вызовах, отвечает на вопрос: как одна функция вызывает другую так, чтобы обе стороны поняли друг друга. На Windows x64 первые аргументы обычно передаются через RCX, RDX, R8 и R9, вызывающая сторона резервирует shadow space, стек должен оставаться выровненным, а часть регистров имеет правила сохранения.

Без ABI невозможно серьезно читать дизассемблерное представление. Инструкция call сама по себе говорит только о переходе. Но ABI объясняет, где искать аргументы перед call, почему рядом есть sub rsp, зачем сохраняются nonvolatile registers и почему эпилог должен восстанавливать стек аккуратно.

Unwind метаданные тоже вырастает из ABI. Windows должна уметь восстановить состояние потока при исключении. Для этого ей нужно знать, что prolog сделал со стеком и регистрами. Поэтому UNWIND_INFO - это не украшение, а сжатое описание действий функции, важных для безопасного движения назад по стеку.

Практическая проверка: в Ghidra открой простую функцию, найди prolog и epilog. Отметь, где меняется RSP, какие регистры сохраняются, и сравни это с Stack size или unwind actions в отчете анализатор.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 6. Prolog, epilog и unwindability

Microsoft x64 documentation использует слово unwindability: код должен быть описуемым для механизма раскрутки стека. Prolog - начало функции, где выделяется кадр стека и сохраняются регистры. Epilog - конец, где состояние возвращается. Для обычного чтения кода это технические детали, но для исключений это жизненно важная карта.

Если исключение произошло внутри функции, Windows должна понять, как перейти к вызывающей функции. Она не может просто угадать старый RSP. Ей нужна UNWIND_INFO. Там записано, сколько байтов выделено на стеке, какие регистры сохранены, где есть обработчик и есть ли chained info.

Аномалии unwind метаданные часто означают, что эта карта странная: не туда указывает RVA, плохо декодируется структура, обработчик выглядит подозрительно, таблица перекрывается, entry не отсортированы или метаданные лежит в необычном месте. Это не теоретические придирки, а вопросы к способности Windows и инструментов корректно понимать функцию.

Практическая проверка: выбери функцию со статусом OK и функцию со статусом Failed. Сравни их Version/flags, Stack, Обработчик и Anomalies/error. Хорошая защита начинается с такого контраста.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 7. RUNTIME_FUNCTION как оглавление функций для Windows

На x64 Windows многие функции описываются строками RUNTIME_FUNCTION. Каждая строка содержит BeginAddress, EndAddress и UnwindInfoAddress. Это похоже на оглавление: откуда докуда функция и где лежит инструкция для раскрутки стека. Но в отличие от обычного оглавления, ошибка здесь влияет на обработку исключений.

Если Begin >= End, диапазон бессмысленен. Если таблица не отсортирована, поиск может стать неоднозначным. Если диапазоны перекрываются, один адрес может претендовать на несколько функций. Если UnwindInfoAddress указывает за пределы образа, анализатор не может безопасно прочитать структуру.

Статический анализ проекта начинается именно с таких инвариантов. Он не пытается доказать намерение автора. Он проверяет договор: таблица должна быть читаемой, диапазоны должны иметь смысл, указатели должны попадать в образ, декодер должен иметь достаточно байтов.

Практическая проверка: открой Функции времени выполнения в отчете и отсортируй взглядом Begin/End. Найди размер функции, секция, unwind RVA и статус. Если есть признак table_integrity, объясни его через конкретную строку таблицы.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 8. UNWIND_INFO как сжатая биография кадр стека

UNWIND_INFO хранит версию, flags, размер prolog, количество unwind codes и дополнительные поля. Если есть EHANDLER или UHANDLER, после unwind codes появляется обработчик RVA. Если есть CHAININFO, структура ссылается на другую RUNTIME_FUNCTION. Эти поля выглядят сухо, но вместе они рассказывают, как функция вошла в свое состояние.

Новичок часто спрашивает: почему нельзя просто посмотреть дизассемблерное представление? Можно, но дизассемблерное представление показывает инструкции, а UNWIND_INFO показывает, что Windows считает официальной картой восстановления. Если эти два взгляда расходятся, это и есть аналитически интересное место.

Проект декодирует UNWIND_INFO осторожно: проверяет границы, версию, flags, выравнивание, количество кодов, наличие хвостовых полей. Поэтому parse failure - это не 'программа не понравилась', а конкретная проблема чтения структуры по правилам формата.

Практическая проверка: найди функцию с EHANDLER или UHANDLER. Покажи обработчик RVA, секцию обработчик и объясни, почему наличие обработчика меняет смысл строки в таблице.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 9. SEH, VEH, UEF и цепочка обработки исключения

Обработка исключений в Windows - это не одна кнопка. Есть структурированная обработка исключений, векторные обработчики исключений обработчики, фильтр необработанного исключения, время выполнения таблицы функций и диспетчер исключений. В проекте особенно важны API AddVectoredExceptionHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable и RtlInstallFunctionTableCallback.

Динамический анализ не заменяет статический. Он отвечает на другой вопрос: происходило ли во время запуска то, что статически выглядело подозрительно или важным? Например, был ли зарегистрирован обработчик, была ли добавлена таблица функций времени выполнения, случилось ли exception event, попал ли точка останова в обработчик.

Если в динамическом отчете Время выполнения table calls = 2, это значит, что процесс реально вызвал API, связанные с динамическими таблицами функций. Если Срабатывания обработчиков > 0, это значит, что выполнение дошло до обработчика. Это сильнее, чем просто наличие адреса в PE, но все равно зависит от выбранного пути запуска и тайм-аут.

Практическая проверка: открой Журнал выполнения и выдели строки с RtlAddFunctionTable, RtlInstallFunctionTableCallback, AddVectoredExceptionHandler или SetUnhandledExceptionFilter. Потом найди соответствующие счетчики в Динамическая проверка.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 10. Статика и динамика как две половины доказательства

Статический анализ читает файл без запуска. Его сила в полноте структуры: можно увидеть секции, таблицы, директории, метаданные и много функций сразу. Его слабость в том, что он не знает, какой путь выполнения реально активируется. Динамическая проверка запускает процесс под наблюдением. Его сила в фактах поведения. Его слабость в неполноте: один запуск не обязан пройти все ветви.

Поэтому лучший вывод выглядит не как 'статика против динамики', а как корреляция. Статика говорит: в файле есть необычное размещение unwind метаданные. Динамика говорит: во время запуска был зарегистрирован обработчик или добавлена таблица функций времени выполнения. Вместе они дают более сильный рассказ: структура необычна, и время выполнения действительно касается механизма исключений.

Если динамика молчит, это не отменяет статическую аномалию. Возможно, путь выполнения не дошел до нужного кода, тайм-аут был коротким, образец требует аргументы или событие возникает только при определенных условиях. В отчете поэтому есть слова уверенность, полнота инструментирования, потерянные события и тайм-аут.

Практическая проверка: для комбинированный образец открой статический HTML и динамический HTML рядом. Найди один и тот же адрес, секцию или обработчик. Сформулируй вывод в две строки: что видно в файле и что подтвердилось при запуске.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 11. Почему признак не равен обвинению

Признак - это обнаруженный признак. Verdict - итоговая оценка. Suspicion оценка - числовая мера подозрительности. Уверенность - уверенность в том, что наблюдения достаточно сильны. Эти слова нельзя смешивать. Один признак может быть важным, но не окончательным. Несколько независимых признак с динамическим подтверждением дают гораздо более сильную позицию.

Хороший реверсер не стремится звучать страшно. Он стремится звучать точно. Вместо 'тут вирус' он говорит: 'в этой функции unwind метаданные находится в writable секция; таблица содержит такой-то обработчик; динамический запуск показал регистрацию такого-то обработчика; поэтому кейс требует ручной проверки и имеет такую-то уверенность'.

Именно поэтому отчеты проекта сохраняют details. Пользователь должен видеть не только карточку Final assessment, но и строки Static признаки, Функции времени выполнения, Журнал выполнения, Manual validation plan. Если отчет нельзя проверить руками, он превращается в черный ящик.

Практическая проверка: выбери один high признак и напиши его как научное утверждение: объект, адрес, нарушенное правило, способ проверки, ограничение вывода. Это лучшая форма ответа на защите.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 12. Ghidra как микроскоп, а не оракул

Ghidra помогает разметить функции, увидеть дизассемблерное представление, вывод декомпилятора, ссылки, sections и карта памяти. Но она не заменяет мышление. Если Ghidra назвала область code, это полезный сигнал, но не математическое доказательство. Если decompiler построил C-подобный код, это удобная реконструкция, а не исходник автора.

Для проекта Ghidra особенно полезна как место ручной проверки representative RVA. Analyzer говорит: проблема около этого адреса. Ghidra позволяет перейти к адресу, увидеть окружающие инструкции, проверить, похожа ли область на функцию, есть ли обработчик, какие ссылки ведут к адресу.

Правильный порядок такой: сначала читаешь отчет и понимаешь, что именно проверяется; потом открываешь Ghidra с конкретным вопросом. Неправильный порядок: открыть Ghidra без вопроса и пытаться глазами найти 'что-то странное' во всем бинарнике.

Практическая проверка: импортируй образец, дождись анализа, перейди к ImageBase + RVA из отчета. Если адрес попал внутрь функции, сравни границы Ghidra с Begin/End из Функции времени выполнения. Если не попал, объясни, почему разметка могла отличаться.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 13. Как читать Журнал выполнения без паники

Журнал выполнения выглядит шумно, потому что отладчик видит много технических событий. CREATE_PROCESS сообщает о запуске процесса. API resolved говорит, что анализатор нашел адрес функции в модуле. BP set говорит, что точка останова установлен. Обработчик hit означает попадание в обработчик. Тайм-аут означает, что наблюдение остановлено по времени.

Не каждая строка - сенсация. Например, сообщение о том, что API resolved but module not present yet, часто означает порядок загрузки модулей, а не провал анализа. Точка останова failures важны, если они не ноль: тогда часть наблюдения могла не сработать. Потерянные события важны, если событий больше, чем collector успел сохранить.

Главное - связывать лог со счетчиками сверху. Exceptions, Срабатывания обработчиков, Время выполнения table calls, Полнота инструментирования, Module покрытие дают быстрый обзор. Журнал выполнения затем объясняет, откуда эти числа взялись.

Практическая проверка: возьми динамический отчет, выпиши пять строк Журнал выполнения и напротив каждой напиши: это событие запуска, установка наблюдения, регистрация API, попадание в обработчик или ограничение эксперимента.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 14. Как устроен pipeline проекта от команды до HTML

Командная строка запускает анализатор с путем к PE и опциями. main собирает конфигурацию, util читает файл безопасно, pe парсер строит модель образа, unwind декодер проходит таблица функций времени выполнения, heuristics engine превращает факты в признаки, dynamic collector при необходимости запускает процесс под отладчик, report writer сохраняет JSON/CSV/HTML.

Эта цепочка важна для доверия к результату. Если отчет пишет 'unwind RVA out of bounds', нужно понимать, на каком этапе это обнаружено. Это не HTML придумал текст. Это результат проверки адреса в PE model и попытки безопасно получить байты UNWIND_INFO.

Разделение слоев помогает отлаживать проект. Ошибка парсер не должна маскироваться как dynamic failure. Ошибка dynamic instrumentation не должна уничтожать статические признак. Ошибка report rendering не должна менять смысл свидетельства.

Практическая проверка: открой исходники src и найди места, которые соответствуют словам парсер, декодер, heuristics, dynamic, report. После этого прочитай один признак в JSON и покажи, какой слой проекта его породил.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 15. Как защищать вывод перед преподавателем

На защите важно не засыпать слушателя терминами, а провести его по маршруту доказательства. Начинай с объекта: какой бинарник, какая команда анализа, какой отчет. Затем назови главный факт: например, необычная unwind секция или время выполнения table registration. Потом покажи адрес и таблицу. Затем объясни, почему это необычно по правилам Windows x64.

Следующий шаг - ручная проверка. Открой Ghidra или HTML, покажи representative RVA, секция, permissions, обработчик или журнал выполнения. Если есть динамика, скажи, что именно произошло во время запуска. Если динамика неполная, честно назови тайм-аут или incomplete instrumentation.

Хороший ответ звучит спокойно: 'я не утверждаю больше, чем вижу; вот байты, вот структура, вот правило, вот наблюдение время выполнения, вот ограничение'. Такая позиция выглядит профессиональнее, чем попытка объявить любой странный бинарник вредоносным.

Практическая проверка: подготовь трехминутный рассказ по одному комбинированный образец. В нем должны быть команда запуска, статический признак, dynamic signal, Ghidra address и финальная осторожная формулировка.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 16. Как читать Microsoft Learn как инженер, а не как словарь

Официальная документация Microsoft обычно написана компактно. Она сообщает структуру, поля, ограничения и поведение API, но не обязана разжевывать путь новичка. Поэтому читать ее нужно не линейно, а с вопросом. Если открыт PE Format, вопрос такой: какое поле помогает мне перейти от байтов файла к образу? Если открыт x64 обработка исключений, вопрос такой: какая структура объясняет строку Функции времени выполнения в отчете?

В проекте документация Microsoft является не украшением библиографии, а основанием инвариантов. PE Format объясняет RVA, VA, sections и каталоги данных. x64 Calling Convention объясняет регистры, стек и теневое место для аргументов. x64 Exception Handling объясняет RUNTIME_FUNCTION, UNWIND_INFO и flags. Debugging События объясняет, почему dynamic collector видит CREATE_PROCESS, EXCEPTION_DEBUG_EVENT и EXIT_PROCESS.

Ошибка новичка - цитировать документацию, не связывая ее с наблюдением. На защите мало сказать 'Microsoft пишет про RVA'. Нужно показать: вот RVA из отчета, вот секция, вот формула перевода, вот почему это поле читается именно так.

Практическая проверка: открой один абзац Microsoft Learn и перепиши его в четыре формы: простыми словами, технически точно, как проверку в анализатор, как вопрос для Ghidra.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 17. Зачем реверсеру Intel manuals, даже если проект про Windows

Windows задает формат PE и правила unwind метаданные, но инструкции выполняет процессор. Поэтому под PE и ABI всегда лежит архитектура: регистры, стек, call, ret, условные переходы, исключения, флаги. Intel Software Developer Manuals нужны не для того, чтобы читать их от первой до последней страницы, а чтобы понимать, где искать строгий смысл инструкции или поведения процессора.

Когда Ghidra показывает дизассемблерное представление, она переводит байты в инструкции. Но значение инструкции все равно определяется архитектурой. Например, изменение RSP важно не потому, что анализатор так решил, а потому что стек и соглашение о вызовах завязаны на RSP. Сохранение регистра важно потому, что ABI определяет, кто должен сохранить значение.

Ошибка новичка - думать, что вывод декомпилятора важнее инструкций. Decompiler помогает читать, но если вопрос касается prolog, epilog, кадр стека или передача управления обработчику, иногда нужно вернуться к дизассемблерное представление и понимать инструкции буквально.

Практическая проверка: в Ghidra выбери функцию, найди первые пять инструкций, и для каждой напиши не перевод на C, а эффект на регистры, стек или управление.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 18. Дизайн эксперимента: почему один запуск не закрывает вопрос

Динамический анализ соблазняет простым ощущением реальности: программа ведь запущена, значит мы увидели правду. На самом деле мы увидели только один путь выполнения при конкретных аргументах, окружении, тайм-аут, правах и состоянии системы. Поэтому динамическая проверка нужно читать как эксперимент, а не как всевидящую камеру.

В EH/Unwind Analyzer это отражается в полях полнота инструментирования, тайм-аут, потерянные события, точка останова failures, module покрытие. Эти поля не второстепенные. Они говорят, насколько хорошо был поставлен эксперимент. Если точка останова failures равны нулю, доверие к наблюдению выше. Если тайм-аут достигнут, нужно честно сказать, что процесс не завершил полный путь.

Ошибка новичка - считать отсутствие срабатывание обработчика доказательством отсутствия поведение обработчика. Возможно, исключение не было сгенерировано, ветка не сработала, программа ждала ввод или образец требует специальных условий.

Практическая проверка: запусти один и тот же образец с разными тайм-аут значениями и сравни Журнал выполнения. Запиши, какие выводы устойчивы, а какие зависят от запуска.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 19. JSON, HTML и CSV как разные формы одного доказательства

HTML удобен для чтения глазами. JSON удобен для точной проверки полей и автоматизации. CSV удобен для импорта в Ghidra или таблицы. Это не три разных результата, а три представления одной модели анализа. Хороший инженер умеет перейти между ними без потери смысла.

Если в HTML видно признак, в JSON должны быть поля, по которым этот признак появился. Если CSV экспортирует begin, end, name или comment для Ghidra, эти значения должны соответствовать тем же RVA и функциям. Когда форматы расходятся, это не косметика, а риск доверия к инструменту.

Ошибка новичка - читать только красивую карточку Final assessment. Карточка нужна для обзора, но доказательство живет ниже: Static признаки, Функции времени выполнения, Журнал выполнения, sections, manual validation plan.

Практическая проверка: для одного отчета найди один и тот же признак в HTML и JSON. Потом, если есть CSV для Ghidra, проверь, какой адрес или комментарий он должен добавить в проект.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 20. Воспроизводимость: команда важнее впечатления

Настоящий технический результат должен воспроизводиться. Поэтому рядом с отчетом всегда должна быть команда запуска анализатор, путь к binary, набор опций, версия инструмента и понимание, debug это сборка или release. Без команды невозможно отличить уверенный результат от случайного скриншота.

В проекте это особенно важно для динамики. Опции --dynamic, --тайм-аут-ms, --html, --json, --csv и путь к образец определяют, что именно будет собрано. Если в презентации показывается динамическая проверка, нужно уметь заново запустить ту же проверку и объяснить, почему появились те же счетчики.

Ошибка новичка - хранить только HTML и забыть команду. На защите преподаватель может попросить повторить анализ. Если команда готова, это выглядит уверенно. Если команда потеряна, даже хороший отчет кажется менее надежным.

Практическая проверка: рядом с каждым показательным отчетом запиши одну строку PowerShell-команды, которая его воспроизводит. Проверь, что пути с пробелами взяты в кавычки.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Модуль 21. Как превратить техническую аномалию в понятный рассказ

Техническая аномалия сама по себе может быть сухой: writable unwind секция, время выполнения table call, срабатывание обработчика, parse failure. Рассказ появляется, когда ты объясняешь, почему это важно, где это видно, как это проверено и чего это не доказывает. Именно этот переход превращает набор признаков в дипломную работу.

Хороший рассказ имеет структуру: объект исследования, нормальное ожидание, наблюдаемое отклонение, статическое доказательство, динамическое подтверждение или его отсутствие, ручная проверка, ограничение вывода. Такая структура понятна даже человеку, который не пишет C каждый день.

Ошибка новичка - начинать с самого сложного термина. Лучше сначала сказать: 'Windows хранит карту восстановления стека. В этом бинарнике часть этой карты лежит необычно и при запуске регистрируется обработчик. Сейчас покажу, где это видно'. После такой фразы слушатель готов к RUNTIME_FUNCTION и UNWIND_INFO.

Практическая проверка: возьми один признак и произнеси его в трех версиях: для одногруппника, для преподавателя, для инженера-реверсера. В каждой версии смысл должен быть один, меняется только глубина терминов.

Контрольный вопрос: можешь ли ты объяснить этот модуль двумя способами - сначала человеку без подготовки, а потом технически точно, используя слова PE, RVA, секция, ABI, UNWIND_INFO, обработчик, статический признак или динамическое событие там, где они действительно нужны? Если оба объяснения сходятся, тема стала рабочей, а не заученной.

Как пользоваться этим маршрутом перед защитой

Перед защитой не надо перечитывать всю книгу подряд в панике. Возьми один показательный бинарник, лучше комбинированный образец, и пройди по модулям как по чек-листу. Сначала назови слой наблюдения. Потом найди PE-секцию и адрес. Потом объясни, что говорит x64 ABI и unwind model. Потом покажи статический признак. Затем открой динамический отчет и проверь, был ли время выполнения signal. В конце сформулируй осторожный вывод.

Если в каком-то месте рассказ ломается, это не провал, а указатель на пробел. Например, если ты не можешь объяснить, почему dynamic тайм-аут не отменяет статический признак, вернись к модулю про статику и динамику. Если путаются адреса, вернись к файловое смещение, RVA и VA. Если Ghidra показывает не то, что ожидалось, вернись к мысли, что инструмент - микроскоп, а не оракул.

Такой способ подготовки хорош тем, что он тренирует не память, а движение мысли. На реальной защите ценят именно это: способность спокойно связать байты, формат, механизм Windows, вывод анализатор и ручную проверку.

Том VI. Практикум: как читать бинарник, отчет и Ghidra как одну историю

Практикум нужен не для количества страниц. Он нужен, чтобы теория стала привычкой. Ниже идут семинарские разборы. Они похожи не формой, а дисциплиной: каждый раз мы начинаем с факта, удерживаем слои отдельно и заканчиваем ручной проверкой.

Главное правило практикума: не пытайся сразу доказать вредоносность. Сначала докажи, что ты правильно понял объект. Где он лежит? Какой у него адрес? Кто его создал? Кто его использует? Какая граница нарушена? Чем это видно во втором инструменте?

Семинар 1. Первое знакомство с PE: байт и pe_section_raw_oob

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - открыть Sections, найти .text, .pdata, .xdata и объяснить роль каждой. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `байт` и признак `pe_section_raw_oob`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `байт` базовая проверка такая: открыть EXE в hex viewer и найти первые байты MZ. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `pe_section_raw_oob` относится к семье `table_integrity`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: сравнить PointerToRawData + SizeOfRawData с размером файла. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 2. Адреса без путаницы: RVA и pe_exception_directory_oob

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - взять один RVA, найти его секция, файловое смещение и VA при известном ImageBase. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это RVA и признак pe_exception_directory_oob.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для RVA базовая проверка такая: найти RVA в отчете и перейти в Ghidra как ImageBase + RVA. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак pe_exception_directory_oob относится к семье table_integrity, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: перевести directory RVA в файловое смещение и проверить size. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 3. RUNTIME_FUNCTION руками: права секции и rf_unsorted

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - выбрать строку таблицы и объяснить begin, end, size, unwind RVA. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это права секции и признак `rf_unsorted`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для права секции базовая проверка такая: найти RW или R-X секцию и объяснить риск. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `rf_unsorted` относится к семье `table_integrity`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: отсортировать begin values и найти нарушение. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 4. UNWIND_INFO без магии: каталог исключений и unwind_reuse_moderate

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - прочитать version, flags, prolog size и понять, есть ли обработчик. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это каталог исключений и признак `unwind_reuse_moderate`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `каталог исключений` базовая проверка такая: сравнить directory RVA/size с секциями. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `unwind_reuse_moderate` относится к семье `unwind_reuse`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: проверить, насколько похожи функции и есть ли другие signals. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 5. Статика против динамики: стек и unwind_writable_section

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - сравнить статический признак с Журнал выполнения и не смешивать их. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `стек` и признак `unwind_writable_section`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `стек` базовая проверка такая: нарисовать stack до call, после call и после prolog. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `unwind_writable_section` относится к семье `metadata_placement`,

поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: проверить W permission у секции. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 6. Ghidra как проверка: исключение и reuse_dominance_combo

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - перейти по representative RVA и посмотреть surrounding дизассемблерное представление. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `исключение` и признак `reuse_dominance_combo`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `исключение` базовая проверка такая: найти first chance и second chance в журнал выполнения. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `reuse_dominance_combo` относится к семье `unwind_reuse`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: посчитать reuse и dominance в одном отчете. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 7. Уверенность без самообмана: динамическая проверка и pe_section_virtual_overlap

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - объяснить, почему высокий оценка не равен доказательству вредоносности. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это динамическая проверка и признак pe_section_virtual_overlap.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для динамическая проверка базовая проверка такая: запустить --dynamic и сравнить Журнал выполнения с static признаки. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак pe_section_virtual_overlap относится к семье table_integrity, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: отсортировать virtual ranges и найти пересечение. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 8. Прочерк в Anomalies/error: Ghidra и rf_oob

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - понять разницу между локальной строкой функции и агрегированным признак. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `Ghidra` и признак `rf_oob`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `Ghidra` базовая проверка такая: импортировать целевая программа и перейти к ImageBase + RVA. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `rf_oob` относится к семье `table_integrity`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: сравнить begin/end с секциями и SizeOfImage. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 9. Тайм-аут в динамике: файловое смещение и unwind_parse_fail

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - объяснить, почему тайм-аут ограничивает наблюдение, но не отменяет статические факты. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `файловое смещение` и признак `unwind_parse_fail`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для файловое смещение базовая проверка такая: вычислить offset = PointerToRawData + (RVA - VirtualAddress). Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `unwind_parse_fail` относится к семье `unwind_integrity`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: открыть bytes UNWIND_INFO и проверить header/codes/tail. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 10. Защитный ответ: секция PE и unwind_in_exec_section

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - сформулировать вывод: факт, адрес, проверка, ограничение. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это секция PE и признак `unwind_in_exec_section`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для секция PE базовая проверка такая: открыть таблицу Sections в HTML-отчете. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `unwind_in_exec_section` относится к семье `metadata_placement`,

поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: проверить секция permissions для Unwind RVA. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 11. Первое знакомство с PE: каталог данных и placement_exec_nonexec_handler_combo

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - открыть Sections, найти .text, .pdata, .xdata и объяснить роль каждой. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `каталог данных` и признак `placement_exec_nonexec_handler_combo`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `каталог данных` базовая проверка такая: найти Exception Directory и проверить его размер. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `placement_exec_nonexec_handler_combo` относится к семье `metadata_placement`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: проверить обе стороны: метаданные и обработчик. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 12. Адреса без путаницы: UNWIND_INFO и pe_section_virtual_oob

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - взять один RVA, найти его секция, файловое смещение и VA при известном ImageBase. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `UNWIND_INFO` и признак `pe_section_virtual_oob`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `UNWIND_INFO` базовая проверка такая: сравнить Version/flags и Обработчик в отчете. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `pe_section_virtual_oob` относится к семье `table_integrity`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: сравнить VirtualAddress + VirtualSize с SizeOfImage. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 13. RUNTIME_FUNCTION руками: ABI и no_runtime_functions

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - выбрать строку таблицы и объяснить begin, end, size, unwind RVA. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `ABI` и признак `no_runtime_functions`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `ABI` базовая проверка такая: объяснить shadow space и 16-byte alignment. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `no_runtime_functions` относится к семье `table_integrity`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: сначала понять, есть ли исполняемый non-leaf код. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 14. UNWIND_INFO без магии: статический анализ и rf_overlaps

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - прочитать version, flags, prolog size и понять, есть ли обработчик. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `статический анализ` и признак `rf_overlaps`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `статический анализ` базовая проверка такая: запустить анализатор без --dynamic. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `rf_overlaps` относится к семье `table_integrity`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: сравнить соседние begin/end после сортировки. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 15. Статика против динамики: уверенность и unwind_reuse_strong

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - сравнить статический признак с Журнал выполнения и не смешивать их. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `уверенность` и признак `unwind_reuse_strong`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `уверенность` базовая проверка такая: объяснить, почему уверенность не равен вероятности malware. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `unwind_reuse_strong` относится к семье `unwind_reuse`, поэтому он

подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: посчитать долю функций с одинаковым UnwindInfoRVA. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 16. Ghidra как проверка: шестнадцатеричная запись и handler_nonexec_section

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - перейти по representative RVA и посмотреть surrounding дизассемблерное представление. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это `шестнадцатеричная запись` и признак `handler_nonexec_section`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для `шестнадцатеричная запись` базовая проверка такая: перевести 0x10, 0x100 и 0x1000 в десятичные числа. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `handler_nonexec_section` относится к семье `metadata_placement`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: сопоставить обработчик RVA с секция permissions. После этого можно идти в Ghidra, но уже не вслепую: у

тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 17. Уверенность без самообмана: VA и writable_metadata_handler_combo

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - объяснить, почему высокий оценка не равен доказательству вредоносности. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это VA и признак `writable_metadata_handler_combo`.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для VA базовая проверка такая: сравнить loaded base в журнал выполнения и адрес обработчик. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `writable_metadata_handler_combo` относится к семье `metadata_placement`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: проверить permissions для обоих RVA. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 18. Прочерк в Anomalies/error: PE и pe_section_misaligned

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - понять разницу между локальной строкой функции и агрегированным признак. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это PE и признак pe_section_misaligned.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для PE базовая проверка такая: найти DOS header, PE signature и таблица секций. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак pe_section_misaligned относится к семье table_integrity, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: проверить адреса и размеры секций на кратность alignment. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 19. Тайм-аут в динамике: RUNTIME_FUNCTION и unwind_rva_oob

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - объяснить, почему тайм-аут ограничивает наблюдение, но не отменяет статические факты. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это RUNTIME_FUNCTION и признак unwind_rva_oob.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для RUNTIME_FUNCTION базовая проверка такая: выбрать строку Функции времени выполнения и объяснить begin/end/unwind RVA. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак unwind_rva_oob относится к семье table_integrity, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: проверить Unwind RVA через секция mapping. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Семинар 20. Защитный ответ: кадр стека и unwind_anomalies

Начнем с простой ситуации. У тебя есть отчет анализатор и бинарник. Тема семинара - сформулировать вывод: факт, адрес, проверка, ограничение. Не открывай сразу все окна подряд. Сначала назови объект исследования: в этой задаче это кадр стека и признак unwind_anomalies.

Первый слой - файл. Здесь важно понять, где физически лежат байты. Для кадр стека базовая проверка такая: найти sub rsp в дизассемблерное представление и объяснить выделение frame. Если на этом этапе перепутать файл и image, все последующие адреса станут шаткими.

Второй слой - PE image. Нужно найти секцию, права, virtual range и связь с exception directory или функции времени выполнения. Признак `unwind_anomalies` относится к семье `unwind_integrity`, поэтому он подсказывает, в какую часть отчета смотреть первой.

Третий слой - человеческая проверка. Для выбранного признака ручная проверка звучит так: сравнить unwind operations с дизассемблерное представление prolog. После этого можно идти в Ghidra, но уже не вслепую: у тебя есть адрес, контекст и вопрос.

Четвертый слой - аккуратный вывод. Формулируй не 'это точно плохо', а 'это необычно по такой причине, подтверждается такими полями, требует такой проверки, ограничение такое'. Именно эта осторожность делает реверс-инженера сильным.

Том VII. Финальный экзамен: четыре сквозных разбора от байта до защиты

Финальная глава нужна для того, чтобы собрать книгу в одно движение. До этого мы отдельно говорили о PE, адресах, ABI, RUNTIME_FUNCTION, UNWIND_INFO, признаках, Ghidra, статике и динамике. Но на практике перед тобой будет не отдельный термин, а бинарник и вопрос: что с ним не так, насколько этому можно верить и как это объяснить другому человеку.

Ниже четыре сквозных разбора. Это не новые признаки и не повтор таблиц. Это тренировка маршрута мышления. Каждый раз мы начинаем с команды и отчета, затем переходим к статическому факту, затем ищем динамическую поддержку, затем открываем Ghidra или ручную проверку, затем формулируем осторожный вывод. Именно так техническое наблюдение превращается в защищаемый результат.

Разбор 1. Статическая аномалия без динамического подтверждения

Представь образец, где статический отчет показывает unwind секция unusual и writable placement, но динамическая проверка дает нули по срабатывания обработчиков. Слабый ответ звучал бы так: 'динамика ничего не нашла, значит аномалии нет'. Сильный ответ другой: статический признак остается фактом структуры файла; динамический запуск лишь не прошел путь, где эта структура проявилась поведением.

В таком разборе первым делом нужно показать таблица секций. Где находится unwind RVA? Какие permissions у секции? Почему это необычно по сравнению с .pdata/.xdata? Затем нужно открыть Функции времени выполнения и показать, какие строки затронуты. Если Anomalies/error заполнен для многих функций, объясни, что признак относится не к каждой функции как отдельному злому действию, а к общей области метаданные, которую используют эти функции.

Динамический отчет читать нужно честно. Если Exceptions = 0, Срабатывания обработчиков = 0, Время выполнения table calls = 0, это не отрицание статического факта. Это описание данного запуска. Проверь тайм-аут, полнота инструментирования, точка останова failures и module покрытие. Если тайм-аут был коротким, скажи это. Если instrumentation incomplete, не делай сильный поведенческий вывод.

Защитная формулировка: 'Статически обнаружено необычное размещение unwind метаданные в такой-то секции с такими-то правами. Динамический запуск не подтвердил достижение обработчика, поэтому я не усиливаю вывод поведением. Ручная проверка заключается в просмотре секция permissions, unwind RVA и соответствующих строк Функции времени выполнения'.

Мини-зачет: сформулируй этот разбор в одном абзаце без потери слоев: файл, PE, адрес, признак, dynamic свидетельства, Ghidra, ограничение вывода. Если хотя бы один слой пропал, вернись к отчету и восстанови цепочку.

Разбор 2. Комбо-кейс: статика видит структуру, динамика видит действие

Лучший демонстрационный пример для защиты - когда static и dynamic говорят об одном механизме с разных сторон. Например, статический отчет показывает подозрительное размещение или обработчик метаданные, а динамический отчет показывает RtlAddFunctionTable, RtlInstallFunctionTableCallback, SetUnhandledExceptionFilter или hit обработчика.

Статическую часть начинай с объекта: имя образец, команда запуска, путь к HTML/JSON. Затем покажи Final assessment, но не останавливайся на карточке. Перейди в Static признаки, найди code, severity, affected count, representative RVA. После этого открой Функции времени выполнения и покажи конкретную строку: begin, end, unwind RVA, секция, обработчик, anomalies/error.

Динамическую часть начинай со счетчиков. Время выполнения table calls > 0 означает, что процесс реально обратился к механизму динамических таблицы функций. Срабатывания обработчиков > 0 означает, что выполнение дошло до обработчика, на который анализатор поставил точка останова. Журнал выполнения дает текстовое доказательство: строки BP set, API call, срабатывание обработчика, exception event, exit процесс.

Защитная формулировка: 'Статика показывает, где в PE лежит подозрительная метаданные, а динамика показывает, что во время выполнения процесс действительно взаимодействует с механизмом исключений/таблицы функций. Поэтому это не только необычная форма файла, но и наблюдаемое время выполнения-поведение в выбранном сценарии запуска'.

Мини-зачет: сформулируй этот разбор в одном абзаце без потери слоев: файл, PE, адрес, признак, dynamic свидетельства, Ghidra, ограничение вывода. Если хотя бы один слой пропал, вернись к отчету и восстанови цепочку.

Разбор 3. Ghidra-проверка representative RVA

Когда в отчете есть representative RVA, это приглашение к ручной проверке. Нельзя защищать адрес только словами. Нужно показать, как перейти к нему в Ghidra. Если ImageBase равен 0x140000000, то RVA 0x1010 соответствует адресу 0x140001010. Если base другой, расчет меняется. Эта арифметика должна быть автоматической привычкой.

В Ghidra сначала проверь Memory Map и ImageBase. Затем используй Go To и введи calculated VA. Если адрес попал в функцию, сравни границы функции с Begin/End из Функции времени выполнения. Если Ghidra не разметила функцию так же, не паникуй: Ghidra строит свою базу анализа, а анализатор читает PE метаданные. Различие само по себе может быть предметом объяснения.

После перехода смотри surrounding дизассемблерное представление. Есть ли prolog? Есть ли изменение RSP? Есть ли переход к обработчик? Есть ли ссылки на область метаданные? Если речь о обработчик RVA, проверь, попадает ли обработчик в executable секция. Если речь о writable метаданные, вернись к секция permissions.

Защитная формулировка: 'Я не прошу верить отчету как черному ящику. Вот representative RVA, вот перевод в адрес Ghidra, вот окружающие инструкции или данные, вот связь с таблицей отчета. Поэтому вывод можно проверить руками'.

Мини-зачет: сформулируй этот разбор в одном абзаце без потери слоев: файл, PE, адрес, признак, dynamic свидетельства, Ghidra, ограничение вывода. Если хотя бы один слой пропал, вернись к отчету и восстанови цепочку.

Разбор 4. Как объяснить ограничения, не ослабляя работу

Ограничения не делают проект хуже. Наоборот, честно названные ограничения делают вывод профессиональным. Статический анализ ограничен тем, что он читает структуру, а не все возможные пути выполнения. Динамический анализ ограничен тайм-аут, входными аргументами, окружением и покрытием. Ghidra ограничена эвристиками разметки и качеством анализа конкретного файла.

В отчете ограничения уже имеют технические следы. Полнота инструментирования показывает, насколько наблюдение завершено. Точка останова failures говорят, удалось ли поставить точки наблюдения. Потерянные события говорят, не потерял ли collector часть событий. Manual validation plan говорит, какие ручные шаги нужны для усиления вывода.

На защите не нужно прятать эти поля. Лучше сказать: 'этот запуск подтверждает такие-то события, но не доказывает отсутствие других путей'. Такая фраза звучит взросло. Она показывает, что ты

понимаешь не только инструмент, но и научную дисциплину проверки.

Финальная формулировка всей книги: 'EH/Unwind Analyzer не заменяет реверс-инженера. Он быстро и воспроизводимо поднимает структурные и время выполнения-факты, связывает их с адресами и отчетами, а человек затем проверяет самые важные места в Ghidra, JSON, HTML и исходном коде проекта'.

Мини-зачет: сформулируй этот разбор в одном абзаце без потери слоев: файл, PE, адрес, признак, dynamic свидетельства, Ghidra, ограничение вывода. Если хотя бы один слой пропал, вернись к отчету и восстанови цепочку.

Разбор 5. Benign-контраст как доказательство аккуратности инструмента

Показывать только подозрительные образцы опасно: слушатель может решить, что анализатор везде находит тревогу. Поэтому рядом с комбинированный образец полезно показать benign образец. Там важно не отсутствие красивой драмы, а спокойная инженерная проверка: парсер прочитал PE, unwind table декодирована, сильных статических аномалий нет, динамическая проверка не нашла необычных обработчик события.

Benign-контраст объясняет границы чувствительности. Если benign report показывает оценка 0 или no strong anomalies, это не значит, что программа 'хорошая во всех смыслах'. Это значит, что именно проверяемые EH/Unwind признаки не дали сильного сигнала. Такой ответ честный и точный.

На защите benign-кейс можно показать в одну минуту. Открой Final assessment, затем Функции времени выполнения с OK-строками, затем Analysis details. Скажи: 'вот нормальная картина, с которой я сравниваю подозрительный образец'. После этого переход к комбинированный образец становится понятнее.

Защитная формулировка: 'Инструмент не обязан кричать на каждый PE. Его ценность в том, что он различает спокойные структуры, структурные нарушения и время выполнения-события. Benign-контраст нужен как контрольная точка эксперимента'.

Мини-зачет: сформулируй этот разбор в одном абзаце без потери слоев: файл, PE, адрес, признак, dynamic свидетельства, Ghidra, ограничение вывода. Если хотя бы один слой пропал, вернись к отчету и восстанови цепочку.

Разбор 6. Когда отчет и Ghidra расходятся

Иногда анализатор и Ghidra показывают разные границы функции или разную уверенность в коде. Это не обязательно ошибка. Analyzer читает PE метаданные и таблица функций времени выполнения. Ghidra строит программу с помощью дизассемблерное представление, анализа потоков управления, эвристик функций и decompiler pipeline. Они смотрят на один бинарник разными методами.

Если Ghidra не размечает функцию там, где RUNTIME_FUNCTION указывает диапазон, нужно задать вопрос: область действительно содержит исполняемый код? попадает ли BeginAddress в .text? есть ли prolog? может ли это быть thunk, padding, намеренно странная разметка или поврежденная метаданные? Ответ появляется только после ручного просмотра.

Если Ghidra размечает функцию шире или уже, чем Функции времени выполнения, это тоже материал для объяснения. Windows unwind метаданные описывает диапазон, важный для exception unwinding. Ghidra пытается восстановить функцию как объект анализа. Эти цели близки, но не тождественны.

Защитная формулировка: 'Расхождение между отчетом и Ghidra я не прячу. Я объясняю источник каждого взгляда: PE метаданные против статической разметки инструмента. Затем вручную проверяю байты, секцию, prolog и ссылки'.

Мини-зачет: сформулируй этот разбор в одном абзаце без потери слоев: файл, PE, адрес, признак, dynamic свидетельства, Ghidra, ограничение вывода. Если хотя бы один слой пропал, вернись к отчету и восстанови цепочку.

Итоговый маршрут перед демонстрацией

Перед демонстрацией выбери один сильный комбинированный образец и один спокойный benign образец. Benign нужен не для красоты, а для контраста: он показывает, что инструмент умеет молчать, когда сильных признаков нет. Комбинированный образец нужен для основного рассказа: он показывает, как статическая аномалия и динамическое событие соединяются в доказательную цепочку.

Для каждого образец сохрани команду запуска, HTML, JSON и при необходимости Ghidra CSV. В HTML отметь карточку Final assessment, таблицу Static признаки, Функции времени выполнения и Журнал выполнения. В JSON проверь точные поля. В Ghidra перейди к representative RVA. После этого проговори вывод вслух. Если рассказ занимает больше пяти минут, убери второстепенные детали, но не убирай адреса и ограничения.

Главная цель финального экзамена - не запомнить этот текст, а научиться двигаться. Сначала факт. Потом структура. Потом время выполнения. Потом ручная проверка. Потом осторожный вывод. Этот порядок превращает даже сложный EH/Unwind материал в понятный инженерный рассказ.

Заключение. Как из этого курса вырастает реверс-инженер

Если после этой книги ты смотришь на EXE уже не как на неизвестный значок, а как на систему договоров, цель достигнута. Компилятор договаривается с ABI. Linker договаривается с PE. Loader договаривается с секциями и relocations. Windows exception dispatcher договаривается с RUNTIME_FUNCTION и UNWIND_INFO. Analyzer читает эти договоры и ищет места, где они нарушены или выглядят необычно.

Реверс-инженер отличается не тем, что знает больше страшных слов. Он отличается тем, что умеет спокойно двигаться от наблюдения к выводу. Он не перескакивает через слои. Он не путает RVA и VA. Он не принимает признак за приговор. Он не считает один динамический запуск полной истиной. Он умеет сказать: "вот факт, вот адрес, вот проверка, вот ограничение".

Этот учебник должен быть не последней точкой, а стартом. Дальше бери реальные отчеты проекта, открывай Ghidra, сравнивай benign и комбинированный образцы, читай исходный код анализатор и постепенно сокращай дистанцию между байтом и смыслом.